

Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN
INFORMATICA

MANIPOLAZIONE CON ROBOT MOBILI TRAMITE TECNOLOGIE OPEN SOURCE

Relatore: Matteo Luperto
Correlatore: Prof. Nicola Basilico
Correlatore: Michele Antonazzi

Elaborato Finale di:
Matteo Vignaga
Matr. Nr. 02305A

ANNO ACCADEMICO 2024-2025

“The shapes are always changing. Changing is their normal state, like us. Even if we’re not changing on the outside, we’re changing on the inside constantly. There’s some stuff about me that I’d been ignoring for a long time. I’m afraid of that stuff. But it’s part of who I am. As long as I know the shape of my soul, I’ll be all right.”

– Jake The Dog

Indice

Indice	i
1 Introduzione	1
2 Tecnologie di base	3
2.1 ROS 2	3
2.2 RViz 2	9
2.3 Robotica e modelli comuni	10
2.3.1 Concetti di base	11
2.3.2 TurtleBot 3	12
2.3.3 Robot quadrupedi	13
2.3.4 Bracci robotici	14
2.4 Simulazione	14
2.4.1 Gazebo Classic/Harmonic	15
2.4.2 Modellazione: SDF, URDF e xacro	16
3 Operazioni di base	18
3.1 Percezione del Robot	18
3.1.1 Odometria	18
3.1.2 Trasformazioni	20
3.2 Teleoperazione	23
3.2.1 Controller ed interfacce hardware	24
3.2.2 Plugin di gazebo	27
4 Operazioni complesse e scenari applicativi	28
4.1 SLAM	28
4.1.1 Localizzazione	28
4.1.2 Mapping	29
4.1.3 Esecuzione di SLAM in ROS	31
4.1.4 Applicazione con Turtlebot3	32
4.2 Navigazione	34

4.2.1	Navigation 2	35
4.2.2	QuadStack	37
4.2.3	Applicazione con Go2	38
4.3	Motion planning	39
4.3.1	MoveIt 2	39
4.3.2	MTC: MoveIt Task Constructor	43
4.3.3	Applicazione con UR5 o Panda	43
4.4	Object detection e pose estimation	43
4.4.1	Applicazione con TurtleBot Home Service Challenge	44
5	Integrazione delle tecnologie	46
5.1	Progettazione dello scenario completo	46
5.2	Sviluppo	47
5.3	Applicazione con turtlebot3_nav_pickandplace	51
6	Conclusioni	55
	Bibliografia	57

Capitolo 1

Introduzione

Questo lavoro di tesi si colloca nell'ambito della robotica mobile, disciplina che riguarda lo studio e la progettazione di robot in grado di eseguire movimenti in autonomia. L'obiettivo di questa tesi è fornire un'analisi di come questi robot si muovono e possono manipolare oggetti, a seguito dell'apprendimento dell'ambiente di lavoro e delle tecnologie utilizzate nello stato dell'arte. Questo obiettivo viene raggiunto fornendo una raccolta strutturata delle informazioni studiate, che potrà essere trattata, di fatto, come una guida introduttiva allo studio, progettazione e sviluppo di applicazioni in ambito robotico Open Source.

Il lavoro è iniziato con uno studio delle tecnologie Open Source e della loro architettura, partendo dal framework industry-standard Robot Operating System 2 (ROS 2) ed il simulatore Gazebo con il sistema di integrazione con le librerie di ROS. Successivamente sono state studiate le operazioni più comuni nei loro aspetti teorici e nelle loro varianti di esecuzione, come la modellazione della percezione dei robot, la teleoperazione manuale, la generazione di mappe, la navigazione autonoma e la pianificazione di movimenti in presenza di ostacoli. Per ognuna di queste operazioni, si è eseguita anche un'applicazione pratica con modelli di robot comuni a livello accademico ed industriale in simulazione, per approfondire ed estendere l'apprendimento in vista di un'implementazione pratica finale. Difatti, il lavoro è stato concluso con lo sviluppo di un progetto che mostra l'applicazione pratica e l'integrazione tra le tecnologie studiate, di fatto estendendo i modelli pubblici disponibili di TurtleBot 3 con Open Manipulator. I requisiti per affrontare questo testo sono pochi; si assume una conoscenza basilare dei concetti di programmazione in Python o in C++, che sono i linguaggi supportati dalle tecnologie oggetto di studio. Anche il funzionamento degli strumenti di build verrà solamente accennato, in quanto è al di fuori del contesto della robotica.

Questo testo, dunque, si occuperà di esporre vari concetti e tecnologie: nel Capitolo 1 viene spiegato l'obiettivo di questo lavoro e la struttura del testo; nel Capitolo

2 vengono presentate le tecnologie che compongono l'ambiente di lavoro *industry-standard* per lo sviluppo, la simulazione e il debugging di tecnologie per robot; verranno inoltre presentati alcuni robot comuni o popolari nel settore accademico e/o industriale. Nel Capitolo 3 verranno spiegate le tecniche di odometria e teleoperazione dei robot, che rappresentano la base delle operazioni da utilizzare con i robot. Nel Capitolo 4 verranno spiegate operazioni più complesse, con dimostrazioni di applicazioni pratiche per ciascuna di esse. Nel Capitolo 5 viene raccontato il lavoro dietro lo sviluppo del package `TB3 Nav PickAndPlace`, prodotto appositamente durante il lavoro di tesi per mostrare l'integrazione delle tecnologie studiate. Nel Capitolo 6 verranno aggiunte alcune informazioni accessorie sull'argomento, come tecnologie aggiuntive che potrebbero essere interessanti da approfondire.

Capitolo 2

Tecnologie di base

2.1 ROS 2

La tecnologia principale alla base del lavoro trattato in questo testo è Robot Operating System (*ROS*) nella sua seconda iterazione, ROS 2. Si tratta di un insieme di librerie e strumenti utili per lo sviluppo software per robot, le cui unità di lavoro sono chiamate nodi e vengono raccolte in pacchetti (spesso si farà riferimento a questi con il termine inglese *package*). Essendo ROS una tecnologia Open Source, anche i pacchetti e le librerie utilizzate con esso lo sono, e verranno trattati i più rilevanti nei capitoli seguenti. ROS 2 è la versione attuale del sistema operativo ed è, a sua volta, divisa in diverse distribuzioni. Al momento della scrittura di questa tesi, le distribuzioni LTS¹ sono ROS 2 Humble Hawksbill (compatibile con Ubuntu 22.04) e ROS 2 Jazzy Jalisco (compatibile con Ubuntu 24.04). Il lavoro pratico di questa tesi è stato sviluppato con la distro humble installata su Ubuntu 22.04. Per l'installazione di ROS, si può seguire il tutorial ufficiale,² avendo cura di selezionare la versione desiderata.

Strumenti di build singola e complessiva

I progetti ROS possono contenere molteplici pacchetti, a loro volta contenenti codice eseguibile che deve passare per strumenti di compilazione. Per questo motivo, il sistema di build può risultare complesso; viene utilizzato il tool di meta build *Ament*, il quale richiede due componenti:

- un tool di **compilazione individuale** per i singoli pacchetti, quali CMake o Python setuptools. Se ne utilizza un wrapper per ciascuno, rispettivamente *Ament CMake* ed *Ament Python*. *Ament CMake* mette a disposizione funzioni

¹Long-Term Support, versione di un software che verrà mantenuta e supportata nel lungo termine. Nel caso delle distribuzioni di ROS, attorno ai 5 anni.

²<https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>

di utilità costruite attorno al popolare strumento di build per progetti C/C++ CMake, che usa il file `CMakeLists.txt` per la definizione delle dipendenze. Ament Python è uno strumento analogo, ma è costruito attorno a Python `setuptools` ed il file `setup.py`, lo strumento di build per librerie Python più popolare al momento.

- un tool di **compilazione complessiva** per l'intero ambiente di lavoro, in questo caso *Colcon*; questo facilita il processo di build con molti package dipendenti tra loro grazie al grafo delle dipendenze generato tramite i file `package.xml` contenuti in ognuno di essi.

Il metodo di build individuale dev'essere dichiarato in fase di creazione del pacchetto, tramite il comando:

```
ros2 pkg create <nome_package> --build-type <build_tool> --license <nome_licenza>
```

Dove `<build_tool>` può essere *ament_cmake* o *ament_python*, mentre la licenza più comunemente utilizzata è *Apache 2.0*, licenza permissiva per lavori Open Source che consente il riutilizzo, la modifica e la distribuzione di prodotti derivati per qualsiasi scopo e senza la necessità di mantenere la stessa licenza (salvo per le componenti non modificate).

Tra le tecnologie per la configurazione dell'ambiente di lavoro si aggiunge anche **rosdep**, uno strumento da riga di comando per l'installazione dei pacchetti ROS di cui si dichiara la dipendenza nel file `package.xml`.

Struttura dell'ambiente di lavoro

I progetti ROS si basano su ambienti di lavoro chiamati *workspace*, e sono essenzialmente directory che contengono i pacchetti all'interno di una cartella `src`, la quale viene utilizzata per la compilazione complessiva da Colcon generando alcune cartelle: `log`, `build` e `install`. I pacchetti rilevanti di un progetto complesso vengono raccolti nella directory `src`, raggiungendo strutture complesse per un ambiente di lavoro più grande: ne viene fornito un esempio in Figura 1, insieme alle cartelle tipiche che si possono trovare all'interno di un pacchetto ROS.

Con l'ambiente completo, può essere necessario installare alcuni pacchetti tramite `rosdep`, il quale leggerà le dipendenze dichiarate nel file `package.xml` ed installerà tutte quelle che trovano un riscontro con quelle presenti nel suo indice.³ È sufficiente usare i seguenti comandi dalla root del setup:

```
rosdep init
rosdep update
rosdep install --from-paths src -y --ignore-src
```

³<https://github.com/ros/rosdistro>, `/<rosdistro>/distribution.yaml` per i package ROS, `/rosdep/base.yaml` per dipendenze apt e `/rosdep/python.yaml` per dipendenze pip.

```

/nome_ws
|--- /log
|--- /install
|   |--- setup.bash           <- script per le variabili d'ambiente
|   '--- ...
|--- /build                   <- destinazione dell'installazione di risorse
'--- /src
    |--- /package1
    |   |--- /src             <- nodi e script C++/Python
    |   |--- /include         <- librerie C++
    |   |--- /launch          <- file di launch
    |   |--- /description     <- rappresentazione del robot
    |   |--- /rviz            <- file di configurazione per RVIZ
    |   |--- /gazebo          <- modelli 3D per la simulazione
    |   |--- /config          <- file di configurazione, generalmente .yaml
    |   |--- /resources       <- altre risorse utili
    |   |--- CMakeLists.txt   <- file per la build con CMake
    |   '--- package.xml      <- file per la build con Colcon e rosdep
    '--- /package2
        |--- /src
        |--- /launch
        |--- /resources
        |--- config.py         <- file per la build con Python setuptools
        '--- package.xml

```

Figura 1: Struttura di un workspace con un pacchetto in C++ ed un pacchetto in Python.

Dove l'opzione `-from-paths` indica la directory da cui cercare i file `package.xml`, `-y` acconsente automaticamente alle installazioni e `-ignore-src` evita l'installazione di package già presenti nell'ambiente di lavoro.

La compilazione di questo workspace avviene tramite il tool di compilazione complessiva **Colcon**, che legge i file `package.xml` di ogni pacchetto e genera il grafo delle dipendenze per permettere un processo di build più semplice ed ordinato. Un'opzione importante da utilizzare durante la compilazione è `-symlink-install`, che permette di creare link a risorse non compilate (come gli script Python) per evitare di dover ricompilare l'intero pacchetto ad ogni modifica.

```
colcon build --symlink-install
```

Questo genererà le cartelle `log`, `build` e `install`. All'interno della directory `install` è presente il file `setup.bash`, necessario per l'overlay di ROS. Tramite il comando `source /install/setup.bash` si potranno eseguire i comandi di ROS 2 sugli eseguibili del progetto. All'interno della directory `build` è presente la cartella `share` per ogni pacchetto dell'ambiente di lavoro, che viene spesso utilizzato per il reperimento delle risorse come file di configurazione, file di launch e modelli 3D tramite i metodi `find_package_share()` delle librerie di Python e C++.

Nodi

I nodi sono l'unità di lavoro di ROS 2, ed è raccomandato svilupparli seguendo il *principio di minima responsabilità*: possono usufruire di molteplici metodi di comunicazione, ma il loro scopo dev'essere unico (es. controllare i giunti del robot, elaborare i dati ottenuti dai sensori, etc.). Per sviluppare queste unità si utilizzano le librerie di ROS per Python (*rclpy*⁴) e per C++ (*rclcpp*⁵). Tramite la classe `Node` delle librerie, si possono istanziare o dichiarare publishers e subscribers per più topic, servers e clients per molteplici service e action, ed anche parametri ottenuti dall'esterno. Per eseguire un nodo è sufficiente utilizzare il seguente comando.

```
ros2 run <nome_package> <nome_nodo> --ros-args -p <nome_arg>:=<valore>
```

Comunicazione tra nodi

L'unità di lavoro di un pacchetto di ROS 2 è il *nodo*: ne segue che un sistema applicativo sarà composto da una rete di nodi in comunicazione tra loro, direttamente tramite actions o services oppure indirettamente tramite topic.

- i *services* permettono una comunicazione client-server per eseguire operazioni su richiesta, con i dati strutturati in file `.srv`;

⁴Documentazione ufficiale: <https://docs.ros2.org/latest/api/rclpy/>

⁵Documentazione ufficiale: <https://docs.ros2.org/latest/api/rclcpp/>

- le *actions* permettono una comunicazione client-server per eseguire operazioni complesse o di lunga durata, dove il server non invia solamente un messaggio finale, ma anche messaggi di feedback intermedi (es. la distanza mancante per un'action di navigazione). I messaggi delle action sono strutturati secondo file `.action`;
- i *topics* permettono una comunicazione multi-a-molti con una semplice politica di publish-subscribe. I messaggi da inviare su un topic devono seguire la struttura specificata nei file `.msg`;

Messaggi standard

La comunicazione tra nodi avviene tramite un'interfaccia definita nel file `.srv`, `.action` o `.msg` di riferimento, che contengono la struttura dei messaggi scambiati. È possibile definire la propria interfaccia di comunicazione, oppure utilizzare una delle interfacce già esistenti e fornite dai vari pacchetti disponibili online. Di seguito, un elenco dei pacchetti di interfacce più comuni, che sono state incontrate durante il periodo di studio.

- **std_msgs**: mette a disposizione tipi di dati primitivi come `Bool`, `Int32`, `Float32`, `String` e `Header`. Quest'ultimo viene utilizzato per i metadata dei messaggi, come timestamp e frame di riferimento per le trasformazioni (di più sull'argomento nella Sezione 3.1.2. Tutti i tipi che contengono un `Header`, in genere, seguono la nomenclatura `<Type>Stamped`.
- **geometry_msgs**: espone una serie di dati per la rappresentazione di forme geometriche primitive.
 - `geometry_msgs/msg/Point`, `Point32`, `PointStamped`: rappresentano un punto in uno spazio tridimensionale tramite 3 numeri a virgola mobile con 64 e 32 bit di precisione.
 - `geometry_msgs/msg/Vector3`, `Vector3Stamped`: rappresentano vettori di spazi tridimensionali tramite 3 numeri a virgola mobile, differendo dai `Point` solamente per significato semantico.
 - `geometry_msgs/msg/Quaternion`, `QuaternionStamped`: rappresentano quaternioni con 4 numeri in virgola mobile da 64 bit.
 - `geometry_msgs/msg/Pose`, `PoseStamped`, `PoseWithCovariance`: rappresentano una posa nello spazio, composta da una posizione (un `Point`) ed un orientamento (un `Quaternion`). La variante `PoseWithCovariance` aggiunge una matrice di covarianza 6x6 per rappresentare l'incertezza del dato.

- `geometry_msgs/msg/Twist`, `TwistStamped`, `TwistWithCovariance`: rappresentano valori di velocità divisi in due vettori, uno per la componente lineare ed uno per la componente angolare.
- `geometry_msgs/msg/Wrench`, `WrenchStamped`: rappresentano espressioni di forza nello spazio, scomposte in due vettori per le componenti lineari ed angolari.
- `geometry_msgs/msg/Transform`, `TransformStamped`: rappresentano la trasformazione tra due sistemi di coordinate tramite un vettore per la traslazione ed un quaternion per la rotazione. La versione `Stamped` non aggiunge solamente l'header, ma anche il nome del sistema di coordinate a cui fa riferimento la trasformazione.
- **`tf2_msgs`**: definisce l'interfaccia dei messaggi da pubblicare sul topic `/tf`, di cui si discuterà il ruolo centrale nella Sezione 3.1.2. L'interfaccia è `tf2_msgs/msg/TFMessage`, che contiene una lista di `geometry_msgs/msg/TransformStamped`.
- **`sensor_msgs`**: standardizza le rappresentazioni dei dati forniti dai sensori.
 - `sensor_msgs/msg/LaserScan`: i dati di un sensore laser vengono rappresentati tramite un Header che contiene il sistema di coordinate di riferimento delle misurazioni (generalmente il sensore stesso ne è l'origine) e vari dati in virgola mobile che descrivono la misurazione; la rappresentazione di ciascun laser segue il senso anti-orario, dove il primo è quello frontale. I dati sulla misurazione sono l'ampiezza del campo visivo, la distanza in radianti tra ciascun laser e la frequenza di misurazione. Infine, una lista di numeri in virgola mobile rappresenta le misurazioni di ciascun laser.
 - `sensor_msgs/msg/Image`: le immagini vengono rappresentate tramite una matrice di numeri interi senza segno a 8 bit (`UInt8[]`), assieme a vari metadati sull'immagine come dimensioni, codifica ed endianness.
 - `sensor_msgs/msg/PointCloud2`: alcuni sensori come le videocamere RGBD possono fornire dati come nuvole di punti, ossia un insieme di punti in uno spazio tridimensionale, a cui può anche essere associato un valore RGB e di intensità. Assieme ai punti vi sono valori come `height` e `width` della nuvola (che assumono le dimensioni dell'immagine oppure `width` è la quantità di punti totale). Gli altri valori sono utili per fare parsing dell'intera lista.
 - `sensor_msgs/msg/Imu`: le misurazioni dell'Inertial Measurement Unit sono una collezione di dati; un Quaternion per l'orientamento, un Vector3 per l'accelerazione lineare, ed un Vector3 per la velocità angolare, assieme alle relative matrici di covarianza.

- `sensor_msgs/msg/JointState`: per rappresentare lo stato di un insieme di giunti, si scompone lo stato in varie liste della stessa dimensione; l'*i*-esimo oggetto della lista di stringhe contiene il nome univoco dell'*i*-esimo giunto, ed allo stesso indice, nelle liste di numeri a virgola mobile `position`, `velocity` ed `effort`, viene memorizzato il valore del giunto corrispondente. Ciascuna lista di valori è opzionale, laddove i giunti non avessero modo di misurare una o più delle 3 grandezze.

File di launch

Quando un progetto fa uso di molti nodi, eseguirli tutti manualmente da riga di comando diventa un'operazione lunga e tediosa, motivo per cui si può automatizzare questo processo tramite i file di launch: script Python che si occupano di generare la *launch description*, ossia una descrizione del processo di inizializzazione e di lancio. Questa comprende un elenco di nodi, altri launch file da eseguire e degli argomenti da passare a questi ultimi. La nomenclatura standard per questi eseguibili è `<nome_operazione>.launch.py`.

```
ros2 launch <nome\_file>.launch.py arg1:=val arg2:=val
```

Un'ottima guida sull'utilizzo dei file di launch è disponibile nel README della repository [1].

Convenzioni

L'ecosistema ROS è Open Source, ed in quanto tale, trae beneficio dalla comunità che si è sviluppata attorno ad esso per continuare a migliorarlo. Per fare ciò, è necessario definire degli standard da seguire per lo sviluppo ed un modo per proporre nuove funzionalità da introdurre. Le *ROS Enhancement Proposals* (REPs) hanno proprio questo scopo, e si possono consultare sul sito ufficiale [2]. Le REPs più utili al fine di svolgere lavori di livello introduttivo verranno menzionate durante il testo, come la REP1 [3], che definisce in maniera esaustiva il sistema di REPs, la loro struttura e le loro tipologie.

2.2 RViz 2

RViz 2 (Robot Visualization, documentazione ufficiale: [4]), comunemente chiamato solo RViz, è uno strumento di visualizzazione 3D molto usato nei progetti ROS, grazie all'ampia gamma di strumenti messi a disposizione. Viene utilizzato non solo per ottenere una vista 3D dei modelli del robot, ma anche come strumento di debug grafico. Le viste messe a disposizione dal tool possono essere salvate in un file di

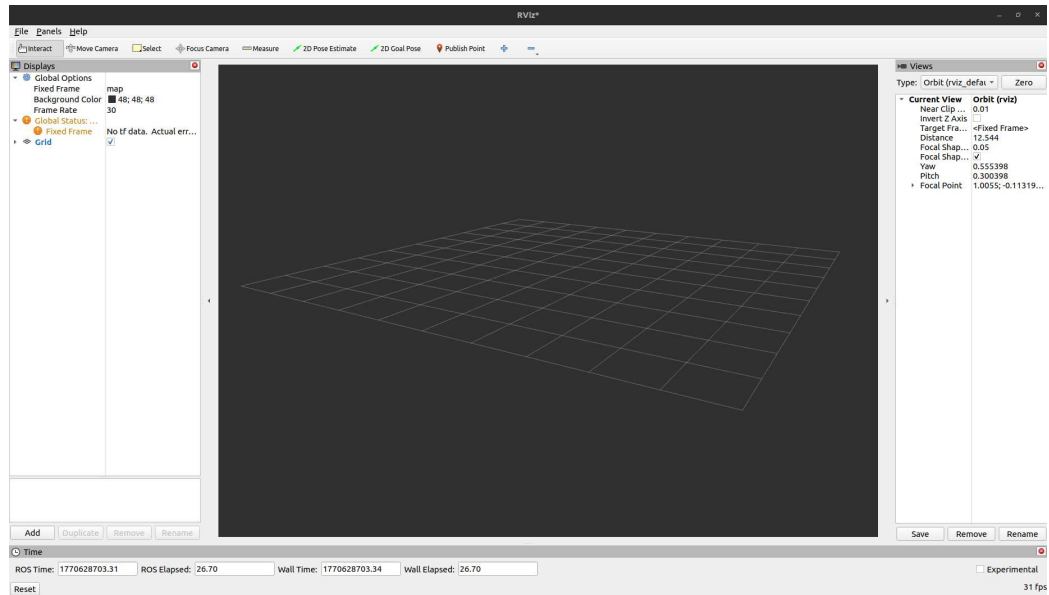


Figura 2: Schermata iniziale default di rviz.

configurazione con l'estensione `.rviz`, che potrà essere utilizzato come parametro in successivi avvii del tool per riottenere il setup completo.

In Figura 2 si può osservare la disposizione degli strumenti: la barra in alto contiene, tra gli altri, strumenti che vedremo durante il Capitolo 4 per la navigazione autonoma (*2D Pose Estimate* e *2D Goal*). A sinistra, invece, vengono situate le viste (i *displays*) che vengono aggiunte tramite il pulsante *Add* e selezionando la funzionalità che si vuole visualizzare: il modello del robot, la percezione di qualche sensore, la struttura del robot, etc. Man mano che verranno trattati elementi utili da visualizzare, verrà menzionato il corrispettivo plugin di RViz.

Per l'installazione non è necessario fare nulla: la sua repo ufficiale fa parte della lista delle repository di ROS 2, quindi verrà incluso automaticamente durante l'installazione del sistema operativo.

2.3 Robotica e modelli comuni

In questa sezione verranno esposti brevemente i concetti basilari di robotica che possono essere utili nella lettura dei seguenti capitoli e di altri testi del settore.

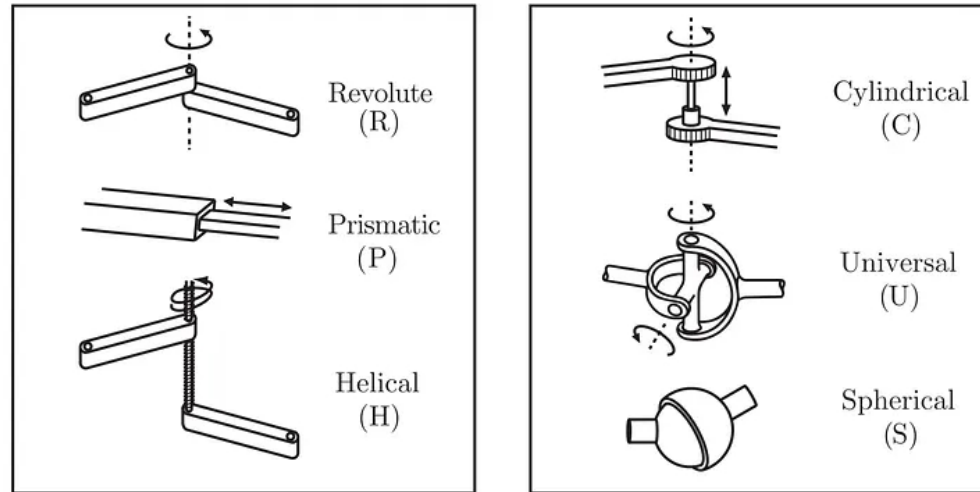


Figura 3: Tipi di giunti, da Modern Robotics di Kevin M. Lynch e Frank C. Park, 2017. [5]

2.3.1 Concetti di base

La robotica è la scienza che si occupa dello studio, della progettazione e dell'operazione dei robot, agenti artificiali attivi il cui ambiente di lavoro è il mondo reale. I campi applicativi dei robot sono molteplici, dall'uso personale a quello industriale, con lo scopo di ridurre, facilitare e ottimizzare il carico di lavoro delle persone.

Un robot generico è composto da un **corpo** rigido principale (convenzionalmente denominato *base link*), collegato ad altri **bracci** rigidi (*links*) tramite **giunti** (*joints*), ed uno strumento finale per svolgere i compiti per cui è progettato (*end effector*). All'interno di questi giunti sono presenti dei codificatori (o *encoders*), che forniscono dati sullo stato del giunto e rendono il movimento dei bracci preciso. I giunti possono essere di varia natura in base al movimento che consentono di eseguire, principalmente rotatorio (rotazione attorno ad un asse) oppure prismatico (traslazione lungo un asse). In Figura 3 vengono illustrati tutti i tipi di giunti.

Ciascun giunto fornisce alcuni *gradi di libertà* (DOF, *degrees of freedom*), ossia una capacità di movimento in relazione ad un asse, come riportato nella Tabella 1. L'architettura complessiva avrà una DOF dipendente dai giunti che la compongono,

Joint type	DOF
Fixed	0
Revolute	1
Prismatic	1
Helical	2
Cylindrical	2
Universal	2
Spherical	3

Tabella 1: Tabella che riporta i degrees of freedom dei tipi di giunti più comuni.

fino ad un massimo di 6 (traslazione lungo i 3 assi e rotazione attorno ai 3 assi).

Oltre ai componenti della struttura di base del robot, sono presenti i sensori e gli attuatori, che gli permettono di interagire con il suo ambiente.

I **sensori** sono quei componenti che permettono al robot di ottenere dati sull'ambiente, tramite i quali potrà essere costruita una sua percezione, ossia un'approssimazione del mondo in cui si trova. I sensori più comuni sono i seguenti:

- IMU (Inertial Measurement Unit): combina un insieme di accelerometri, giroscopi e magnetometri per misurare l'accelerazione lineare ed angolare del robot, implementando una propriocezione del suo movimento;
- LiDAR (Light Detection and Ranging) e SoNAR (Sound Navigation and Ranging): misurano la distanza da eventuali ostacoli, fornendo una percezione accurata tramite un insieme di sensori laser o sonori disposti attorno al robot;
- videocamere: possono essere installate videocamere di diverso tipo, che possono essere utilizzate per migliorare la percezione del robot, come videocamere termiche, videocamere RGBD, etc.;
- sensori di forza: generalmente posti tra l'ultimo link del braccio e l'end effector, per eseguire movimenti ottemperati;
- sensori tattili: permettono al robot di afferrare un oggetto con la forza corretta in base alla fragilità dello stesso, e misurano vibrazioni per evitare scivolamenti;

2.3.2 TurtleBot 3

I robot della piattaforma TurtleBot⁶ sono particolarmente usati in ambito accademico per lo studio e la ricerca, sviluppati da ROBOTIS per fornire modelli di piccole dimensioni ed economici per l'apprendimento. La piattaforma fornisce anche un insieme di

⁶Sito ufficiale: <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>

pacchetti ROS⁷ per operazioni di generazione di mappe, navigazione e manipolazione, sia tramite un modello reale sia in simulazione.

I robot di TurtleBot sono stati sviluppati in 4 versioni numerate da TurtleBot 1 a TurtleBot 4, ma la versione standard al momento è la terza, che fornisce due modelli: TurtleBot3 Waffle Pi e TurtleBot3 Burger, entrambi dotati di una Raspberry Pi come SBC⁸ e svariati sensori come un IMU, un LiDAR ed una videocamera, come visualizzabile in Figura 4.

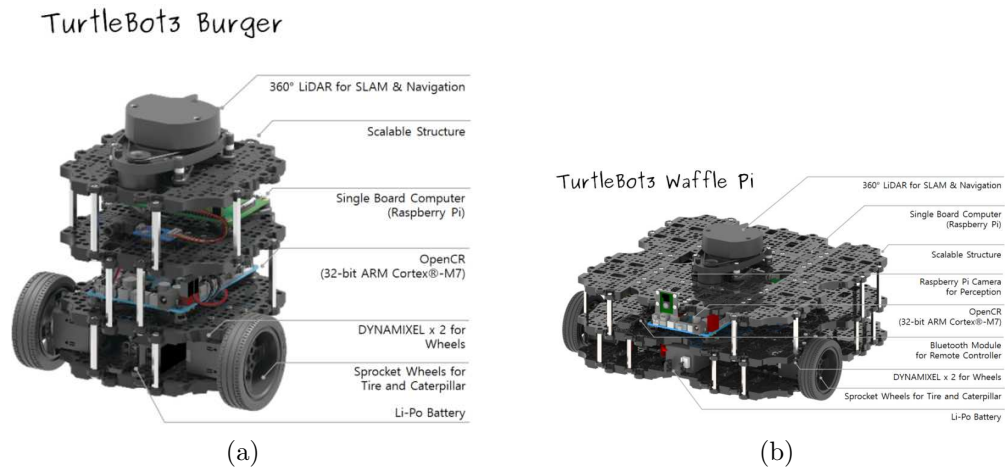


Figura 4: Composizione dei due modelli di TurtleBot 3, fonte: [6].

2.3.3 Robot quadrupedi

I robot quadrupedi sono il frutto di avanzamenti tecnologici nell'ambito dei robot mobili, basati sullo studio della locomozione degli animali a quattro zampe, simulata tramite bracci a tre giunti. Il design a quattro gambe, visualizzabile in Figura 5, rende questi agenti maggiormente complessi, ma anche versatili: sono in grado di muoversi su terreni instabili e con maggiore precisione, rendendoli un'alternativa efficace in ambito industriale (possono muoversi in spazi a 2.5 dimensioni), di soccorso (su terreni disastriati e instabili) ed agricolo (possono spostarsi nei campi senza rovinare il raccolto).

Per questo lavoro di tesi sono state utilizzate simulazioni dei modelli di robot quadrupedi di Unitree⁹ e MAB Robotics¹⁰.

⁷<https://github.com/ROBOTIS-GIT/turtlebot3>

⁸Single Board Computer

⁹<https://www.unitree.com/>

¹⁰<https://www.mabrobotics.pl/>

I modelli di Unitree che sono stati adoperati sono l'A1, l'A2, il Go1 ed il Go2. I modelli di MAB Robotics utilizzati sono l'Honey Badger e il Silver Badger. La scelta dei modelli non è casuale: sono quelli supportati out-of-the-box da QuadStack, lo stack di navigazione autonoma per robot quadrupedi trattato nella Sezione 4.2.2.



Figura 5: Modello Go2 di Unitree.

2.3.4 Bracci robotici

I modelli più popolari per i bracci robotici volti alla manipolazione di oggetti sono quelli della linea di robot a 6 DOF URx di Universal Robots¹¹, la prima azienda ad aver sviluppato un robot collaborativo, o *cobot*, ossia robot in grado di operare in sicurezza in ambienti condivisi con esseri umani.

2.4 Simulazione

Per cominciare ad apprendere la robotica non è necessario acquistare un robot, considerando che anche i modelli più economici che sono stati menzionati fin'ora hanno un range di prezzo che parte dalle centinaia di euro; si possono utilizzare simulazioni grazie a software specifici come Gazebo ed Isaac Sim. Durante il lavoro di tesi sono state utilizzate simulazioni in Gazebo Classic.

¹¹<https://www.universal-robots.com/it/>



Figura 6: Modello UR5e di Universal Robots.

2.4.1 Gazebo Classic/Harmonic

Uno dei simulatori Open Source più popolari per contesti pratici di ROS è Gazebo, che a causa delle nomenclature utilizzate nelle diverse versioni può sembrare confusionario per chi si avvicina al mondo della robotica simulata:

- Gazebo Classic (o Gazebo-11.xx) è la versione legacy del simulatore. Ha raggiunto lo stato di End-Of-Life a gennaio 2025, ma è supportato fino alla versione ROS 2 Humble Hawksbill ed Ubuntu 22.04. Nonostante non avrà ulteriori aggiornamenti, molte risorse online fanno ancora riferimento a questa versione, incluso il lavoro di questo elaborato. L'ambiente di lavoro consigliato con Gazebo Classic è Ubuntu 22.04 Jammy Jellyfish con le installazioni di ROS 2 Humble Hawksbill e un simulatore tra Gazebo Classic e Gazebo Garden.
- Gazebo è la versione moderna, frutto di un fork del repository originale nel 2017, e presenta prestazioni migliori ed una migliore integrazione con i framework di ROS. Il nome Gazebo viene utilizzato in maniera generica come riferimento al software di questo fork, di cui esistono varie versioni codificate in ordine alfabetico, come Gazebo Fortress, Gazebo Garden, Gazebo Harmonic. L'ambiente di lavoro consigliato con Gazebo è Ubuntu 24.04 Noble Numbat con le installazioni di ROS 2 Jazzy Jalisco e la versione di Gazebo Harmonic.

Per altri tipi di ambienti di lavoro è possibile consultare la REP 2000¹² sulla compatibilità dei rilasci di ROS con le varie piattaforme di interesse.

2.4.2 Modellazione: SDF, URDF e xacro

Il simulatore di Gazebo è in grado di renderizzare entità di cui si fornisce la descrizione in un formato basato su XML: SDF (*Simulation Description Format*) oppure URDF (*Universal Robot Description Format*). Di seguito, una breve spiegazione dei formati:

- SDF: il formato ufficiale supportato da Gazebo per la composizione di robot ed oggetti per popolare l'ambiente. Quando si vuole definire un modello in questo formato, è necessario creare una directory con il nome dell'oggetto, contenente un file `model.sdf` ed un file `model.config`. Il primo file contiene la composizione dell'entità tramite un tag principale `<model>` e, al suo interno, una serie di tag `<link>`, `<sensor>` e `<joint>` per descrivere il robot o l'oggetto.

Ogni elemento `<link>`, a sua volta, contiene 3 elementi necessari per il simulatore: `<inertial>`, `<visual>` e `<collision>`. Il tag `<inertial>` viene utilizzato per simulare la fisica dell'ambiente, dove l'inerzia tridimensionale viene elencata con una matrice d'inerzia¹³. Il tag `<visual>` contiene descrizioni sull'aspetto del link tramite i tag `<geometry>` per la forma e `<material>` per l'aspetto. Il tag `<collision>` definisce la forma del link da utilizzare per il calcolo delle collisioni: per evitare di sprecare risorse computazionali, durante simulazioni banali è fortemente consigliato utilizzare forme primitive che approssimano adeguatamente la forma fornita tramite `visual`.

Ogni elemento `<joint>` deve specificare il tipo di giunto di cui si tratta, oltre a un *parent link* e un *child link*, definendo una struttura ad albero del modello.

Ogni elemento `<sensor>` differisce in base al tipo di sensore che definisce, dunque si rimanda il lettore alla documentazione ufficiale¹⁴ per maggiori informazioni sui sensori supportati.

È anche possibile aggiungere dei *plugin* tramite il tag `<plugin>`, utili per fornire funzionalità aggiuntive da librerie esterne, come quelli forniti da Gazebo stesso per robot a guida differenziale.

Il secondo file, `model.config`, contiene una serie di metadati che descrivono l'entità.

¹²<https://www.ros.org/reps/rep-2000.html>

¹³Per chi non fosse avvezzo alla fisica e al calcolo dell'inerzia per ogni singolo componente, si consiglia di utilizzare uno dei vari calcolatori online per oggetti approssimativamente simili a ciò che si vuole modellare.

¹⁴<https://sdformat.org/spec/1.12/sensor/>

Si può vedere un esempio di questi file nel repository del package TB3 Nav PickAndPlace¹⁵, che verrà trattato successivamente nel Capitolo 5, seguendo il path `/gazebo/models`.

- URDF: è il formato per la descrizione dei robot standard per ambienti di lavoro ROS, primo tra tutti RViz. L'architettura del robot è la medesima, una struttura ad albero composta di `<link>` e `<joint>`, ma all'interno di un tag `<robot>`; quando si vuole aggiungere funzionalità utili a Gazebo come sensori e plugin, è sufficiente inserirle dentro ad un tag `<gazebo>`. Anche per questo formato si possono trovare esempi online in vari repository, come quelli di TurtleBot 3, generalmente nelle directory o pacchetti denominati `*_description`. Per scrivere la propria descrizione URDF di un robot, rimane utile consultare la documentazione ufficiale¹⁶.
- xacro: permette di scrivere XML tramite delle macro, utili per generare delle descrizioni *dinamiche* in base ad argomenti e costrutti if-else e *modulari* grazie alla composizione tramite altri file xacro.

Per utilizzare un mondo o un oggetto specifici in simulazione, Gazebo Classic mette a disposizione una variabile d'ambiente `GAZEBO_MODEL_PATH` (oppure `GZ_SIM_RESOURCE_PATH` per Gazebo) a cui attribuire come valore il path della cartella contenente tutti i modelli.

¹⁵https://github.com/MatteoVignaga/Turtlebot3_Nav_PickAndPlace

¹⁶Documentazione di ROS 1 valida anche per ROS 2: <https://wiki.ros.org/urdf/XML>

Capitolo 3

Operazioni di base

Le operazioni elementari per un agente robotico sono semplici: ottenere una stima del mondo esterno e di sé stesso all'interno di esso, e fornire il controllo dei propri movimenti ad attori esterni (un operatore umano o un nodo che gli fornisce comandi a fronte di calcoli specifici).

La *percezione* del mondo esterno si ottiene tramite i sensori, già trattati nel capitolo precedente, ma approfonditi nelle sezioni seguenti; mentre la percezione di sé stesso avviene tramite l'odometria e le trasformazioni della propria descrizione secondo i movimenti dei giunti.

Il *controllo dei movimenti* di un robot può essere ottenuto tramite i controllori (o *controller*), componenti dedicati all'interazione con l'hardware del robot in modo autonomo, tramite meccanismi di retroazione o esponendo un'interfaccia standard per molteplici giunti dello stesso tipo. Quest'ultima implementazione permette di eseguire operazioni come l'operazione a distanza del robot.

3.1 Percezione del Robot

3.1.1 Odometria

L'odometria di un robot è l'operazione di stima della configurazione dello chassis nell'ambiente tramite i dati ottenuti dai sensori, nel tempo. Si tratta di una tecnica che rientra in un insieme di operazioni di localizzazione basate sul calcolo della posizione tramite dati di velocità e movimento, più ampiamente definite come *dead reckoning*; il termine odometria è utilizzato quando si ha a che fare con robot su ruote, ma spesso vengono usati come sinonimi.

Sensori per odometria

I sensori coinvolti sono chiamati sensori odometrici, e sono tutti quelli che possono misurare il movimento del robot:

- I wheel encoders: collegati all'albero delle ruote di un robot a guida differenziale, possono stimare la posizione del robot in base alla quantità di giri effettuati nel tempo ed alla dimensione delle ruote. Non forniscono una stima perfetta, in quanto richiederebbe un contatto delle ruote con il terreno costante e privo di scivolamenti, accumulando errore nel tempo (*drifting*).
- L'Inertial Measurement Unit (IMU) è in grado di stimare posizione ed orientamento del robot tramite calcoli di integrazione della sua velocità lineare ed angolare, ma anch'essa accumula errore con il tempo; dunque, va supportata con altri sensori.
- La VIO (o *Visual-Inertial Odometry*) tramite videocamere o sensori LiDAR permette di ottenere ulteriori stime dei movimenti del robot in base ai movimenti di alcuni punti chiave identificati e tracciati tra i frame catturati. Integrando le stime ottenute con i sensori precedenti si può migliorare ulteriormente la precisione.

La localizzazione all'aperto viene generalmente rafforzata tramite GPS [7], ma non è una soluzione valida per ambienti al coperto: tra le alternative esistono sistemi basati su Wi-Fi o 5G, ma la soluzione più appropriata sarebbe un sistema chiuso e robusto, che non necessiti di infrastrutture esterne. Per progettare questo tipo di sistema e minimizzare l'errore dei singoli sensori menzionati, spesso risulta conveniente utilizzare più sensori contemporaneamente ed unire le stime per avere una stima ancora più precisa. La fusione dei sensori può avvenire in maniera *tightly coupled* o *loosely coupled*. La metodologìa più accurata ma anche più computazionalmente costosa è quella *tightly coupled*, dove i dati dei sensori vengono fusi direttamente all'interno di un algoritmo di stima dello stato. La tecnica di fusione *loosely coupled*, invece, lavora con i dati derivati dai sensori, come valori di posizione o velocità ottenuti tramite una fase di processing intermedia. L'algoritmo principale utilizzato per la fusione (sia *loosely* sia *tightly coupled*) è il *filtro di Kalman esteso* (EKF): un metodo di stima non lineare di transizioni di stato del robot diviso in due passi: il *prediction step* che utilizza osservazioni passate e matrici di covarianza per calcolare una stima dello stato corrente, che viene modificata nell'*update step* tramite le misurazioni correnti ed un coefficiente che descrive il rapporto tra covarianza della stima e delle misurazioni.

Smoothing dell'odometria

Eseguire la fusione dei sensori per la localizzazione 2D o 3D in ROS è semplificato grazie al package *robot_localization*.¹ Si tratta di un sistema *general-purpose* in quanto supporta un'ampia varietà di sensori, *scalabile* perchè permette la fusione di una quantità illimitata di sensori e *flessibile* grazie al supporto di un'ampia varietà di messaggi standard ROS e la possibilità di scegliere, per ogni sensore, quali campi del messaggio concorreranno alla stima [8].

Utilizzare *robot_localization* all'interno del proprio ambiente di lavoro richiede di installare il pacchetto ed eseguire il nodo *ekf_node*, che implementa il filtro di Kalman esteso, con un file di configurazione. La configurazione è un file YAML che contiene i parametri per l'algoritmo Extended Kalman Filter e, per ogni sensore, la configurazione del topic su cui pubblicano i messaggi e la matrice che definisce quali campi dei messaggi utilizzare.

3.1.2 Trasformazioni

Ciascun elemento presente nell'ambiente ROS ha modo di definire la posizione di altri oggetti in base alla propria. Questo tramite il sistema di coordinate (*frame*) dell'elemento, la cui origine è il centro dell'elemento stesso. Questo concetto si estende ad ogni componente *link* del robot, dove la posizione di ogni componente (ad esempio, l'end effector del braccio) può essere espressa tramite un vettore differente in base al frame di riferimento. Da qui sorgono alcuni concetti che verranno definiti in questa sezione: i *frame di coordinate* e le *trasformazioni*.

Frame di coordinate (REP 103 e 105)

Dal momento che i valori di posizione sono ampiamente utilizzati durante la comunicazione tra vari nodi dell'ecosistema ROS, si è ritenuto necessario stabilire una convenzione per l'architettura di riferimento di questi valori. Questo standard viene definito all'interno della REP 103² e della REP 105.³ La prima definisce:

- Il *sistema di coordinate*: viene utilizzata la regola della mano destra, dunque l'asse x "punta" di fronte, l'asse y "punta" verso sinistra e l'asse z "punta" verso l'alto. Esiste un'eccezione per le videocamere, che spesso dispongono di un frame denominato con suffisso o prefisso "*optical*", e segue un sistema di coordinate differente (z verso davanti, y verso il basso, x verso destra).

¹Documentazione ufficiale: https://docs.ros.org/en/noetic/api/robot_localization/html/index.html

²<https://www.ros.org/reps/rep-0103.html>

³<https://www.ros.org/reps/rep-0105.html>

- Le *unità di misura standard* per ogni dimensione misurata: vengono adottate le unità di misura del Sistema Internazionale delle Unità di Misura (SI).

La REP 105 arricchisce lo standard da seguire, illustrando le convenzioni di nomenclatura e il significato semantico dei frame di coordinate più comuni in ROS.

- Il frame *base_link*: è il punto di accesso alla struttura del robot, e può essere posizionato in qualunque posizione rispetto al corpo principale (generalmente, il centro dello chassis).
- Il frame *odom*: è un frame di coordinate la cui origine è fissata alla posizione iniziale del robot. Viene aggiornato continuamente tramite i valori dell'odometria del robot e, dunque, soffre dello stesso problema di drifting; ne segue che l'origine tende a spostarsi nel tempo, risultando in un frame di riferimento globale poco attendibile nel lungo periodo.
- Il frame *map*: è un frame di coordinate la cui origine è fissata al centro della mappa che l'agente ha memorizzato o generato, ed ha lo scopo di essere il frame di riferimento globale nel lungo periodo. Viene aggiornato tramite una funzionalità di localizzazione tramite sensori, un'attività computazionale più complessa che porta ad aggiornamenti periodici e "salti" della posizione del robot relativa all'origine, motivo per cui non è attendibile come riferimento locale nel breve periodo.
- Il frame *earth*: viene utilizzato per permettere a robot che operano in frame map differenti di interagire tra loro secondo un sistema di coordinate Earth Centered Earth Fixed (ECEF). Quando il frame map è uno solo, non è necessario che questo frame sia presente.

Nel sistema di frame del robot, ciascun frame è messo in relazione agli altri secondo una struttura ad albero, come illustrato in Figura 7. Una posizione espressa all'interno di uno qualsiasi di questi frame di coordinate può essere trasformata in modo tale da conoscere la medesima posizione relativa agli altri frame.

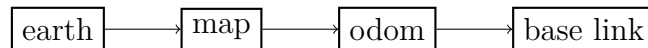


Figura 7: Struttura ad albero dei frame di coordinate secondo REP 105.

Inoltre, è possibile vedere la struttura ad albero dei frame in uno spazio tridimensionale grazie alla vista TF di RViz, come in Figura 9.

Trasformazioni e tf2

Utilizzare un valore posizionale con un frame di coordinate specifico comporta, talvolta, la necessità di convertirlo in un valore posizionale con un frame di coordinate differente (insomma, "cambiare il punto di vista" della posizione). Basti pensare al caso delle operazioni di *object detection* e *collision avoidance*: la posizione degli oggetti viene inizialmente espressa in relazione al sensore che li rileva; dunque, il frame di riferimento è quello del sensore. Per evitare un ostacolo, però, l'interesse è sapere dove non spostare il corpo principale. Sarà dunque necessario effettuare una *trasformazione*, traslando l'origine del sistema di riferimento a quella del `base_link`. La trasformazione tra due frame viene rappresentata semplicemente tramite la traslazione e la rotazione necessarie.

Questa operazione viene gestita dal package *tf2*, ormai divenuto lo standard per le applicazioni di ROS. Questo pacchetto mette a disposizione un'ampia gamma di strumenti per la visualizzazione dell'albero delle trasformazioni (tramite il nodo *view_frames* di *tf2_tools*, esempio in Figura 8), visualizzazione delle singole trasformazioni (tramite il nodo *tf2_echo* di *tf2_ros*), e strumenti per la pubblicazione delle trasformazioni (publishers e broadcasters). Oltre ai suddetti strumenti, sono state sviluppate anche le relative librerie per C++ e Python, che permettono di eseguire tutte queste operazioni all'interno di un nodo personalizzato, con particolare rilevanza per la classe `Buffer` della libreria *tf2_ros*, fornita del metodo `transform<T>(T in, string target_frame)` per trasformare un valore posizionale relativo ad un frame qualsiasi in un valore posizionale relativo a *target_frame*, a patto che esista una catena di trasformazioni valida tra i due sistemi di coordinate.

Una catena di trasformazioni da un frame ad un altro è valida se e solo se, per ogni frame padre intermedio, esiste una trasformazione valida con il frame figlio. In altre parole, dovranno essere eseguite tutte le trasformazioni lungo il percorso tra i due frame sull'albero delle trasformazioni. Ogni trasformazione dev'essere definita e pubblicata con il relativo messaggio *tf2_msgs/msg/TFMessage* sui topic `/tf` o `/static_tf`; quest'ultimo è un topic utilizzato per trasformazioni statiche o raramente aggiornate (come trasformazioni tra link collegati da giunti fissi), pubblicandole raramente o perfino una sola volta per ridurre il carico computazionale. Generalmente, le trasformazioni principali vengono pubblicate automaticamente da alcune fonti:

- La trasformazione da `base_link` al frame di altri link del robot viene fornita da *transform broadcaster* personalizzati oppure dal package *robot_state_publisher* con l'aggiunta di un *joint_state_publisher*. Il joint state publisher pubblica lo stato di ogni giunto sul topic `/joint_states`, che verrà letto dal robot state publisher assieme alla descrizione URDF del robot per comporre la posa assunta.

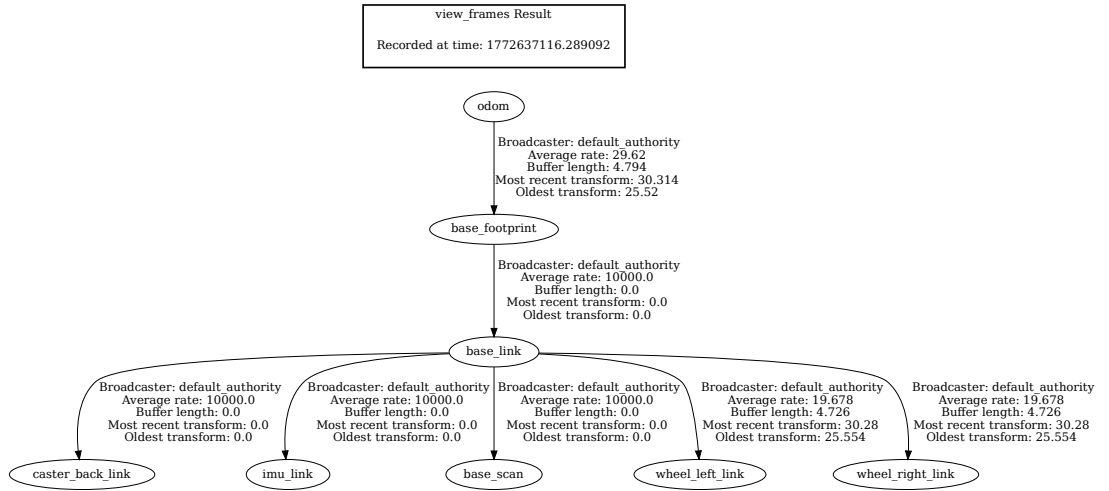


Figura 8: Visualizzazione dell'albero delle trasformazioni prodotta dal nodo `view_frames` di `tf2_tools`. NB: non è presente il frame map perchè non sono stati eseguiti nodi con funzionalità di mapping.

Questo nodo pubblica le trasformazioni sui topic `/tf` e `/tf_static`, e pubblica anche la descrizione URDF trasformata sul topic `/robot_description`.

- La trasformazione da `odom` a `base_link` viene fornita da un nodo che si occupa delle fonti odometriche, come *robot_localization*, *Nav2* o i *plugin di Gazebo* (come il plugin di differential drive).
- La trasformazione da `map` a `odom` viene pubblicata dal nodo che si occupa della localizzazione globale, come *slam_toolbox* e *Nav2*.
- La trasformazione da `earth` a `map` è, solitamente, una semplice trasformazione statica.

3.2 Teleoperazione

La teleoperazione è il primo meccanismo per comandare i movimenti di un agente robotico. Consiste nell'utilizzo di un dispositivo specifico per inviare comandi ai controller dei giunti motori; in ambienti ROS sono spesso utilizzati tastiere e joystick.

La comunicazione tra il dispositivo di comando ed il robot avviene tramite i controller, che devono comunicare con i driver e le interfacce hardware degli attuatori

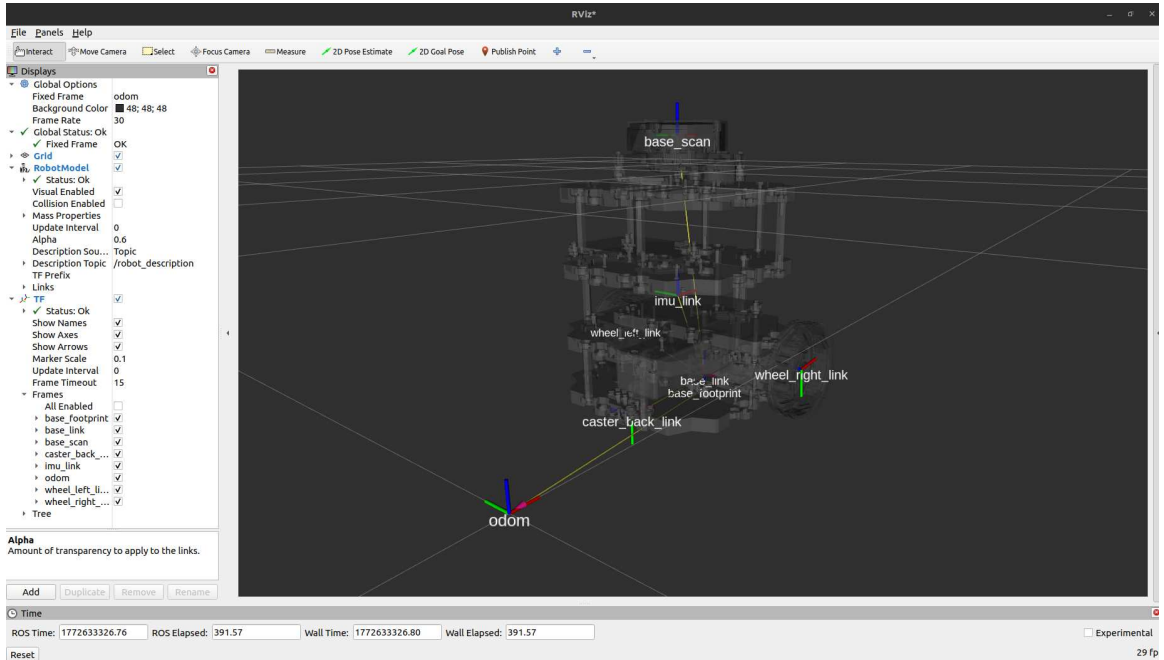


Figura 9: Visualizzazione grafica del sistema di trasformazioni del TurtleBot 3 Burger in RViz.

montati sul robot; e tramite i nodi di ROS, che traducono i messaggi dei driver in messaggi standard ROS e li pubblicano. In questo modo si ha una comunicazione full duplex per il controllo del robot e la ricezione di feedback dai sensori.

3.2.1 Controller ed interfacce hardware

I controller sono algoritmi che definiscono la modalità di controllo di un attuatore; le interfacce hardware permettono di comunicare con le componenti fisiche. Attuatori dello stesso tipo, però, espongono interfacce differenti, richiedendo controller programmati su misura. Questo comporterebbe uno spreco non indifferente di tempo e risorse per la risoluzione di un problema già risolto in altri modi: è per questo che si è ritenuto necessario standardizzare la comunicazione tra controller ed driver, ed è il compito del package *ros2_control*.

L'interfaccia hardware, quando si opera con robot reali, viene fornita dal produttore. Dal momento che sono specifiche per ogni attuatore e sensore, non è compito di questo testo scendere nei dettagli, ma di seguito viene descritta l'astrazione generica per comprenderne il funzionamento. I driver espongono interfacce di comando (*command interfaces*) ed interfacce di stato (*state interfaces*). Le interfacce di comando sono la componente che può essere controllata tramite operazioni di lettura e

di scrittura; le interfacce di stato, come suggerisce il nome, forniscono informazioni sullo stato dell'attuatore e consentono solamente operazioni di lettura. Ad esempio, la ruota di un robot equipaggiata con un wheel encoder potrebbe essere comandata tramite comandi di velocità; in questo caso, sarà presente un'interfaccia di comando per la velocità ed alcune interfacce di stato in base al tipo di encoder (per monitorare velocità, posizione, momento torcente o altro).

ROS 2 control

Per rimediare al problema di specificità delle interfacce hardware, ROS 2 control mette a disposizione due intermediari per la comunicazione: il *resource manager* ed il *controller manager*.

Il resource manager raccoglie tutte le interfacce hardware dalla descrizione URDF del robot e le espone insieme, senza distinzione tra il tipo di dispositivo a cui fanno riferimento. Il package può leggere le interfacce grazie all'URDF perchè viene inserito un tag apposito, `<ros2_control>`, in cui vengono elencate le varie interfacce hardware disponibili.

Il controller manager è un nodo fornito da `ros2_control` ed ha il compito di prendere ciascun controller da avviare ed accoppiarlo con le interfacce di comunicazione di cui ha bisogno. Così, il compito dei controller può essere ridotto a leggere i comandi di controllo pubblicati sui topic, eseguire i calcoli necessari per generare il messaggio da mandare alle interfacce di comando, e pubblicare i feedback delle interfacce di stato. Dal momento che i controller non dipendono più dall'hardware, il package `ros2_control` fornisce una sua estensione, denominata `ros2_controllers`, che mette a disposizione l'implementazione dei controller più comuni, facilmente configurabili tramite file YAML.

Una volta inserite le interfacce hardware nell'URDF e preparato il file YAML con i controller configurati, si può operare con il package in vari modi: eseguendo il nodo del controller manager ed il nodo per lo spawn dei controller; scrivendo un nodo personalizzato in cui si istanzia il controller manager, chiamando i services messi a disposizione dal package, oppure utilizzando gli strumenti da riga di comando di `ros2_control`.

Gazebo ROS 2 control

Il package `gazebo_ros2_control` è un adattamento dell'architettura di `ros2_control` al simulatore. Consiste nell'aggiunta di un plugin `libgazebo_ros2_control.so` per istanziare un nodo di Gazebo che istanzia il controller manager e lo rende in grado di comunicare con il simulatore. Per le nuove versioni di Gazebo il pacchetto è stato rinominato `gz_ros2_control`, ed anche il plugin ha seguito la stessa nomenclatura rimpiazzando `gazebo` con `gz`; il funzionamento rimane il medesimo.

ROS 2 controllers

Di seguito, un elenco esaustivo dei controller più comuni forniti da ros2 controllers.

- Il *joint_state_broadcaster*: non si tratta di un controller, bensì di un *broadcaster*; non comanda nulla, ma legge tutte le interfacce di stato del robot e riporta lo stato dei giunti sui topic `/joint_states` e `/dynamic_joint_states`. Utile come sostituto del joint state publisher per fornire dati al robot state publisher.
- Il *diff_drive_controller*: prende comandi di velocità pubblicati sul topic `/cmd_vel` (o `/cmd_vel_unstamped`) e li traduce nei comandi per le ruote di un robot a guida differenziale. In aggiunta, pubblica dati odometrici sul topic `/odom`, le trasformazioni delle ruote su `/tf` e i comandi di velocità adeguati ai limiti definiti della configurazione sul topic `/cmd_vel_out`.
- Il *forward_command_controller*: un controller base che inoltra i comandi pubblicati sul topic `/commands` direttamente ai giunti di cui si occupa. I comandi vengono organizzati in una lista dove l'i-esimo valore è il comando per l'i-esimo giunto.
- I *position_controllers*: implementazione del *forward_command_controller* con specificità rispetto ai comandi, che rappresentano la posizione che devono assumere i giunti.
- I *velocity_controllers*: implementazione del *forward_command_controller* con specificità rispetto ai comandi, che rappresentano la velocità dei giunti.
- I *effort_controllers*: implementazione del *forward_command_controller* con specificità rispetto ai comandi, che rappresentano la forza che devono esprimere i giunti.
- Il *joint_trajectory_controller*: estensione del *forward_command_controller* che permette di controllare ed interpolare il movimento di un insieme di giunti per seguire una traiettoria definita da una sequenza di punti, che saranno le tappe del percorso da seguire. Supporta interfacce hardware per posizione, velocità, accelerazione e forza applicata. I comandi possono essere mandati tramite topic `/joint_trajectory` oppure tramite action `follow_joint_trajectory`.
- Il *pid_controller*: implementa un sistema di controllo a feedback tramite tecnica PID.

Il topic `/cmd_vel`

In questo Capitolo sono stati trattati tutti i concetti necessari per l'implementazione di un sistema di teleoperazione di un robot a guida differenziale tramite i controller ed i plugin per la guida differenziale, che leggono messaggi di tipo Twist sul topic `/cmd_vel`. A pubblicare questi messaggi ci pensano nodi esterni che si occupano di direzionare i movimenti del robot, come il nodo *teleop_twist_keyboard* dell'omonimo pacchetto per la teleoperazione manuale e *Nav 2* per la navigazione autonoma. Possono capitare, però, eventi in cui sarebbe conveniente rendere il sistema di guida autonoma più flessibile, permettendo ad un operatore umano di intervenire nel controllo robot. In questo caso, è necessario ordinare le fonti da cui arrivano i messaggi di tipo Twist, ed è proprio l'obiettivo del package *twist_mux*: tramite una configurazione YAML, è possibile elencare le fonti dei comandi di velocità ed ordinarle secondo valori di priorità; così il nodo svolge il ruolo di multiplexer per pubblicare solamente i messaggi della fonte con la precedenza. In questo modo, si può dare alta priorità all'operatore umano, il quale avrà la precedenza sul sistema di navigazione autonoma.

Attenzione: in molti casi, potrebbe essere necessario rimappare i topic di uscita delle fonti, affinché non pubblichino direttamente su `/cmd_vel`.

3.2.2 Plugin di gazebo

Anche il simulatore Gazebo mette a disposizione un elenco di controller per sensori ed attuatori comuni, facilmente integrabili all'interno del proprio modello da simulazione tramite i *plugin*.⁴ L'utilizzo dei plugin di Gazebo al posto dei controller di `ros2_control` limita l'utilizzo al solo simulatore, ma semplifica la struttura rimuovendo la necessità di istanziare il controller manager, definire le interfacce hardware, e configurare i controller; essenzialmente, è una buona soluzione per progetti semplici e prototipi per testing.

⁴Lista completa: https://docs.ros.org/en/rolling/p/gazebo_plugins/generated/index.html

Capitolo 4

Operazioni complesse e scenari applicativi

In questo capitolo vengono trattate le operazioni comuni nell'ambito della robotica mobile, che si basano sui concetti trattati precedentemente. Per ogni operazione, oltre alla teoria di fondo, vengono discusse le tecnologie Open Source che ne permettono l'applicazione ed una sezione contenente un'applicazione pratica eseguita durante il lavoro di studio. Le tecnologie trattate sono in larga parte quelle utilizzate nel progetto finale discusso nel Capitolo 5, o semplicemente ritenute rilevanti nel caso di studio.

4.1 SLAM

La prima operazione di cui vale la pena discutere è quella di *Simultaneous Localization and Mapping (SLAM)*, che ha lo scopo di permettere all'agente robotico di creare una mappa dell'ambiente circostante e posizionarsi al suo interno. Dal momento che si tratta di due operazioni distinte, di seguito si propongono le definizioni individuali, per poi discutere l'utilità di svolgerle congiuntamente.

4.1.1 Localizzazione

Assumendo di avere a disposizione una mappa dell'ambiente, è possibile confrontarla con la percezione dell'ambiente esterno ottenuta dai sensori ed ottenere la propria posizione sulla mappa. In questo modo, si ottiene un valore posizionale relativo ad un frame globale (*map*), che può essere utilizzato per varie operazioni come la navigazione autonoma. I metodi di localizzazione principale si basano su sistemi ed algoritmi odometrici già discussi nella Sezione 3.1.

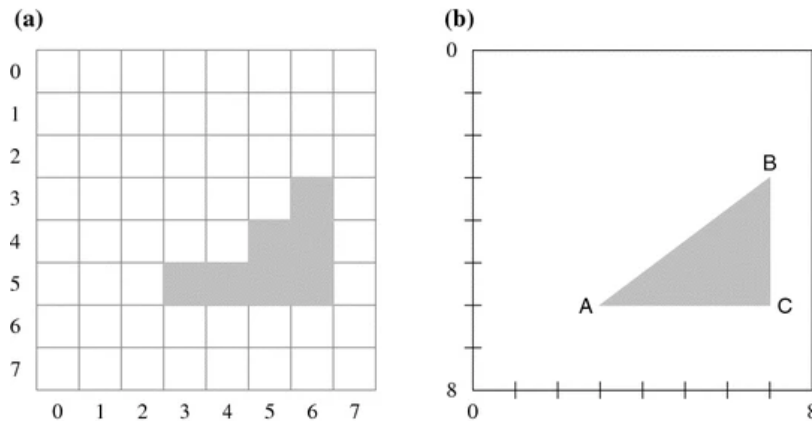


Figura 10: Mappa discreta (a sinistra) e mappa continua (a destra), che rappresentano lo stesso oggetto triangolare. Immagine tratta da [9].

4.1.2 Mapping

La generazione della mappa è un'operazione basata sulla percezione del mondo esterno: conoscendo la propria posizione globale ed osservando le caratteristiche di ciò che circonda il robot, è possibile creare una rappresentazione bidimensionale o tridimensionale del mondo secondo varie tecniche, da quelle basate su grafi ad algoritmi probabilistici più complessi. In questa Sezione viene trattato una tecnica probabilistica per la generazione di mappe a griglia di occupazione, la più comune in progetti ROS ed utilizzata dalle tecnologie di SLAM studiate.

Le mappe generate dai robot possono essere discrete o continue: le *mappe discrete* dividono l'ambiente in celle, mentre gli ostacoli vengono rappresentati bloccando l'insieme di celle che rappresentano l'area occupata; le *mappe continue* rappresentano gli ostacoli con dei punti che definiscono i vertici dell'area occupata. Se ne può vedere una dimostrazione in Figura 10.

La medesima Figura mostra chiaramente che una mappa discreta è più imprecisa: la risoluzione è direttamente proporzionale alla quantità di celle e, viceversa, una cella definisce l'area più piccola che si può considerare occupata, anche quando un ostacolo è considerevolmente più piccolo. Una griglia più fine richiede più memoria e risorse computazionali, notoriamente limitate nei robot mobili. Nonostante ciò, sono più utilizzate perché anche le mappe continue hanno i loro svantaggi; la loro precisione richiede la memorizzazione di molti più vertici man mano che gli oggetti nell'ambiente aumentano, ed i calcoli richiesti quando si tratta di oggetti curvi sono dispendiosi.

La categoria di mappa discreta più adottata in ambienti ROS è la **mappa a griglia di occupazione** (Occupancy Grid Map). Come le mappe geografiche utilizzano i colori per rappresentare il territorio, le occupancy grid map utilizzano valori numerici secondo un codice, che definisce quando la cella è occupata o meno. La codifica più

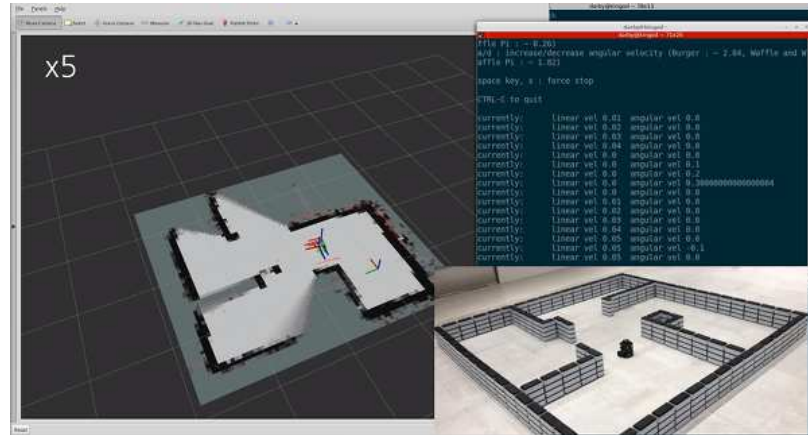


Figura 11: Generazione in tempo reale di una mappa da parte di un TurtleBot. Immagine tratta da [10].

semplice utilizza un bit: 1 per celle occupate, 0 per celle libere; ma dal momento che i sensori da cui si ottengono questi dati sono imprecisi, è più comune utilizzare *modelli probabilistici*, in cui ad ogni cella è associato il valore della certezza che al suo interno sia presente un ostacolo. Successivamente, la griglia può essere filtrata per ottenere la codifica semplice seguendo dei valori di soglia per determinare come contrassegnare ciascuna cella. La mappa viene salvata come un'immagine in formato PGM (*Portable Gray Map*), come visualizzabile in Figura 12, con un file YAML accessorio che definisce metadati come i valori di soglia sopracitati.

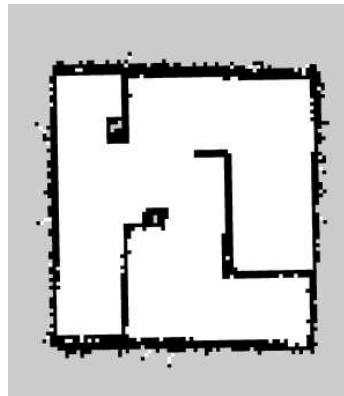


Figura 12: Mappa generata in formato Portable Gray Map.

4.1.3 Esecuzione di SLAM in ROS

All'interno degli ambienti di lavoro ROS, la componente per l'esecuzione di simultaneous localization and mapping è fondamentale e può essere integrata nel proprio progetto grazie ai vari package Open Source disponibili. Tra questi, lo standard è il package *Slam Toolbox*, ma verrà accennato anche *Cartographer*, uno degli standard precedenti ma non più mantenuto dallo sviluppatore originale, Google.

Cartographer [11]

Cartographer è un sistema che mette a disposizione funzionalità di SLAM per ambienti 2D e 3D, grazie all'integrazione dell'omonima libreria C++ con ROS. È in grado di generare occupancy grid maps tramite alcuni nodi messi a disposizione dal package *cartographer_ros*, quali *cartographer_node* e *cartographer_occupancy_grid_node*, ed un file di configurazione in formato LUA. Utilizza algoritmi basati su grafi ed il risolutore Google Ceres, che lo rendono una soluzione molto efficiente e robusta, ma è stato abbandonato a causa della cessazione del mantenimento e della sua complessità eccessiva. Si è ritenuto necessario includerlo perché viene ancora utilizzato come package per SLAM da TurtleBot 3 il quale, come accennato nel Capitolo 2, è molto utilizzato a livello accademico.

Slam Toolbox [12]

La tecnologia di SLAM standard per ambienti ROS è Slam Toolbox: sviluppata da Steve Macenski, grande contributore del panorama Open Source di ROS, è la soluzione per operazioni di SLAM robuste, anche per la generazione di mappe di larghissima scala (fino a 24.000 m²) in tempo reale.

I requisiti per l'utilizzo di Slam Toolbox nel proprio progetto sono i seguenti:

- L'installazione del relativo package ROS *slam_toolbox*;
- L'introduzione di un componente specifico nella descrizione del robot: il link *base_footprint*, che rappresenta essenzialmente la proiezione del robot sul terreno, spesso approssimabile ad un cerchio. Non è necessario inserire alcuna componente inerziale, visiva o di collisione, in quanto la forma verrà fornita nei file di configurazione.
- Un sensore che fornisca dei messaggi di tipo *LaserScan*, come un LiDAR o altri tipi di sensore con dei nodi di conversione appropriati.
- Un file di configurazione dei nodi di mapping e localizzazione in formato YAML. Con l'installazione, si ottiene anche una serie di file di configurazione di default (per le installazioni su Ubuntu, si trovano sotto */opt/ros/<distro>/share*

`/slam_toolbox/config`). Al suo interno vengono specificati valori come i nomi dei frame di riferimento, il topic su cui vengono pubblicati i messaggi LaserScan, e la modalità di esecuzione (*mapping* o *localization*). La modalità *mapping* può risultare un nome fuorviante, perché è la modalità per SLAM, mentre la modalità *localization* si limita a ciò che implica il nome.

Tramite i file di launch del package si può scegliere la modalità di SLAM appropriata per le esigenze del progetto, nello specifico si può scegliere tra:

- `online_async_launch.py`: permette di svolgere operazioni di SLAM in real-time (online), ma senza forzare l'elaborazione di ogni singola scan dei sensori (async).
- `online_sync_launch.py`: permette di svolgere operazioni di SLAM in real-time (online) elaborando ogni singola scan dei sensori (sync).
- `offline_launch.py`: permette di svolgere operazioni di SLAM tramite log di dati precedentemente registrati, come rosbags.
- `localization_launch.py`: inserendo il percorso del file di una mappa nel file di configurazione, mette a disposizione la funzionalità di localizzazione pura.

Una volta generata la mappa, Slam Toolbox mette a disposizione una serie di servizi per salvarla e serializzarla, ma è più comune utilizzare il plugin di RViz *SlamToolbox-Plugin* come visibile in Figura 13, oppure il nodo di Nav2 *map_saver_cli*. Salvare la mappa genera i file PGM e YAML, mentre serializzarla consiste nel generare tutti i file contenenti i metadati necessari per Slam Toolbox per riprendere la mappatura in seguito e continuare a rifinire il risultato, in una funzionalità definita *lifelong mapping*.

4.1.4 Applicazione con Turtlebot3

Durante il lavoro di studio sono state eseguite molte operazioni di SLAM con diverse simulazioni di robot; prima tra tutte per rilevanza accademica, quella in simulazione con un modello TurtleBot 3 equipaggiato con Cartographer, come visualizzabile in Figura 14. Al momento della scrittura di questa tesi, è disponibile un tutorial completo sul sito ufficiale di Robotis¹ per le distro di ROS Humble, Jazzy e Noetic. Consiste nell'esecuzione di alcuni comandi da terminale, spiegati di seguito:

1. Innanzitutto, replicare l'ambiente di lavoro in una workspace, clonando i repository necessari alla simulazione riportati sul sito ufficiale e compilando tramite Colcon secondo quanto spiegato nella Sezione 2.

¹https://emanual.robotis.com/docs/en/platform/turtlebot3/slam_simulation/

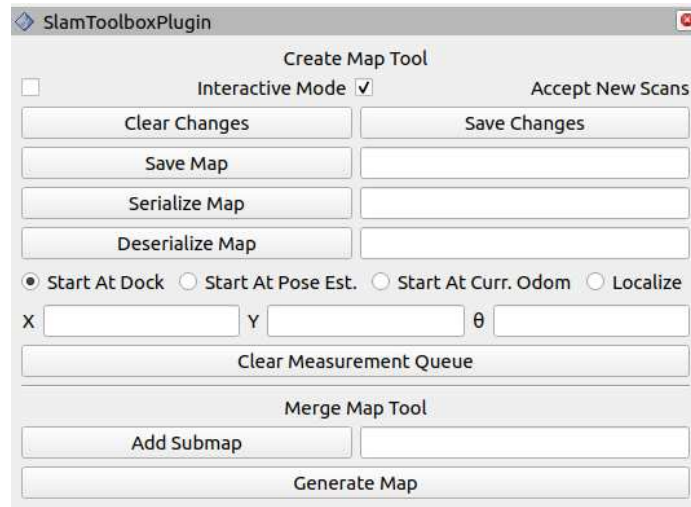


Figura 13: Pannello di SlamToolboxPlugin in RViz.

2. Scelta del modello da spawnare tramite la variabile d'ambiente. Le opzioni disponibili sono *burger*, *burger_cam*, *waffle* e *waffle_pi*.

```
export TURTLEBOT3_MODEL=burger
```

3. Spawn del robot in simulazione, in uno dei mondi forniti dal package di simulazione. Inoltre, è possibile utilizzare un mondo qualsiasi modificando il path del mondo all'interno del file di launch e fornendo il path dei modelli alla variabile d'ambiente, come menzionato nella Sezione 2.4.

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

4. In un secondo terminale, si può procedere all'inizializzazione del nodo di Cartographer tramite l'apposito file di launch.

```
ros2 launch turtlebot3_cartographer cartographer.launch.py
↪ use_sim_time:=True
```

5. In un terzo terminale, l'esecuzione del nodo di teleoperazione per muovere manualmente il robot e generare la mappa. È consigliabile muovere il robot lentamente ed in maniera esaustiva per tutto il mondo, al fine di generare una mappa corretta ed affidabile.

```
ros2 run turtlebot3_teleop teleop_keyboard
```

6. Una volta generata la mappa completa, si può procedere al salvataggio della mappa tramite il map server di Nav 2, e specificando il nome dei file PGM e YAML che verranno generati con l'opzione `-f`.

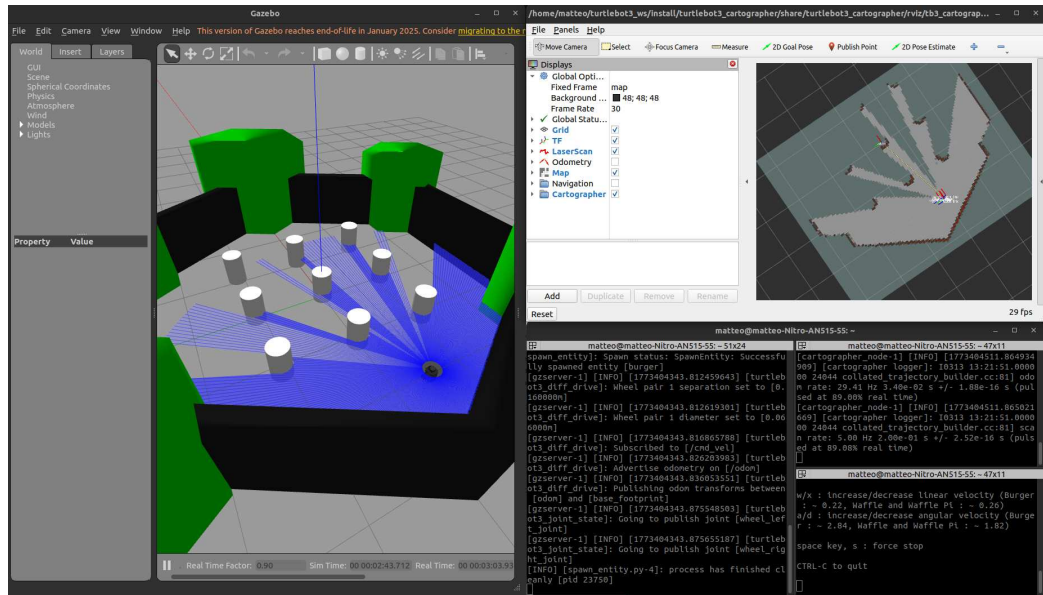


Figura 14: Ambiente di lavoro durante l'esecuzione di SLAM con TurtleBot 3 e Cartographer.

```
ros2 run nav2_map_server map_saver_cli -f ~/map
```

4.2 Navigazione

Il problema della navigazione autonoma è un argomento ampiamente discusso fin dai primi agenti robotici intelligenti; qualsiasi robot definito intelligente dovrebbe essere in grado di muoversi senza il bisogno di operatori umani. Nel corso del tempo, le soluzioni trovate sono state molteplici e seguono all'incirca lo stesso flusso di lavoro: percepire l'ambiente circostante, pianificare il percorso e percorrerlo. Di seguito, una breve analisi teorica delle architetture adottate da queste soluzioni [13]:

- La navigazione deliberativa (*centralized*): si crea un modello globale dell'ambiente tramite sistemi di sensori o input da parte di un utente. Basandosi su questi dati, l'agente ricerca il percorso ottimale per raggiungere il goal. Dal momento che questo metodo di navigazione richiede il modello del mondo prima di qualsiasi altra cosa, presenta alcune carenze quando l'ambiente è sconosciuto o dinamico. Inoltre, la stretta sequenzialità delle operazioni (sensing, planning, action) rende le tecniche deliberative poco robuste: se un modulo fallisce, porta con sé l'intero sistema.

- La navigazione reattiva (*behavior-based*): questo tipo di architettura genera comandi da impartire agli attuatori in base alla percezione corrente, togliendo la necessità di avere un modello del mondo iniziale. I dati sensoriali vengono direttamente mappati in azioni tramite delle funzioni chiamate *task achieving modules/behaviors*. In generale, sono soluzioni più flessibili e meno costose, dal momento che non serve memorizzare un modello del mondo e non c'è pianificazione complessa, ma introducono il problema di coordinazione dei behavior e la mancanza di pianificazione potrebbe rendere difficile l'esecuzione di task complesse.
- La navigazione ibrida: sviluppata come connubio tra le due architetture precedenti, sfrutta la capacità pianificativa della navigazione deliberativa con la flessibilità in ambienti sconosciuti o dinamici della navigazione reattiva. A sua volta è un sistema classificato in 3 stili: manageriale, a gerarchie di stato e model-oriented.
 - Nello stile manageriale, il modulo deliberativo pianifica ad alto livello, poi il piano viene eseguito dal modello reattivo.
 - Nello stile a gerarchie di stato il modello deliberativo usa un record di stati passati per fare path planning, mentre il modello reattivo completa le azioni correnti (passato, futuro e presente).
 - Nello stile model-oriented, si costruisce il modello dell'ambiente circostante come strumento per il modello reattivo, di fatto riducendo il tempo di computazione del modello deliberativo.

Ciascuna architettura migliora la precedente cercando di colmare i casi d'uso che non erano coperti, dunque le tecnologie adottate ad oggi (come anche quelle adottate durante il periodo di studio) seguono un'architettura ibrida.

4.2.1 Navigation 2

Lo stack di navigazione Navigation 2 è la soluzione proposta da Steve Macenski per una tecnologia di navigazione autonoma integrata a ROS 2 [14]. È basata sul *ROS Navigation Stack*, una tecnologia open source proposta nel 2010 che è stata utilizzata su larga scala da aziende e ricercatori. Nonostante ciò, basandosi su ROS 1, privo di performace elevate e garanzie di sicurezza, non è la soluzione più efficiente possibile.

Alla base di Navigation 2 ci sono due concetti fondamentali: le *layered costmaps* ed i *behavior tree*. Le *costmap* sono una rappresentazione dell'ambiente costruita tramite la mappa ed alcune informazioni aggiuntive fornite da molteplici algoritmi intermedi (i *layers*) prima che venga passata ai pianificatori del percorso. I *Behavior Trees* sono un'alternativa alle macchine a stati finiti per sistemi modulari, così come le

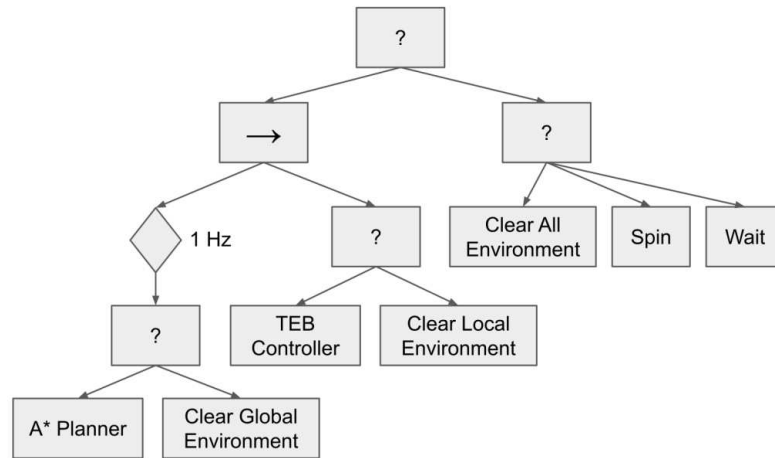


Figura 15: Behavior tree utilizzato durante il test The Marathon. [14]

reti di Petri lo sono per sistemi concorrenti [15]. In Figura 15 si può vedere un esempio di behavior tree utilizzato come configurazione durante i test di [14]: i nodi "?" sono nodi di fallback, trasferiscono il contesto di esecuzione ai figli in ordine finché uno di loro non termina con successo; il nodo "→" è un nodo sequenziale ed esegue tutti i figli in sequenza, terminando con successo se e solo se tutti i figli terminano con successo. Questi sistemi di modellazione del comportamento sono configurabili tramite un file in formato XML, strutturando ed organizzando operazioni di *pianificazione*, *controllo* e *recupero* senza bisogno di programmare. Ciascuna operazione viene eseguita da un server che implementa un'interfaccia a *plugin*, per permettere di creare o selezionare algoritmi differenti.

Anche gli algoritmi e le tecniche standard utilizzati sono progettati per essere i più appropriati in ambienti fortemente dinamici:

- *Behavior Trees* per la modellazione di comportamenti complessi, dividendoli in comportamenti primitivi modulari. Il Behavior Tree Navigator è il componente principale e si occupa di caricare il file XML di configurazione e di istanziarne i relativi nodi.

Sono di particolare interesse i comportamenti di recovery, inseriti per la gestione di situazioni di errore in una subtask o nell'intera operazione di navigazione. Queste azioni di recupero possono essere inserite all'interno dell'albero per la risoluzione di vari problemi; è raccomandato inserirle in ordine di aggressività dell'approccio. Le più comuni sono un semplice reset delle costmap, lo spin del robot per districarsi da posti stretti e l'attesa (wait). È importante notare che l'utilizzo di operazioni di recupero non è da interpretare negativamente, in quanto parte di un sistema fault-tolerant.

- *Dynamic Window Approach* (DWA) per pianificazione e collision avoidance.
- Un sistema di *layered costmaps* per la percezione dell'ambiente, dove ogni layer aggiunge o estende informazioni sulla mappa prima che venga passata ai pianificatori. I layer predefiniti sono:
 - lo *static layer*: layer statico popolato tramite la mappa fornita dalla tecnologia di SLAM adottata.
 - l'*inflation layer*: arricchisce la mappa aggiungendo un'area di buffer attorno agli ostacoli, dove viene aumentato il costo di percorso tramite una funzione di decadimento esponenziale. In questo modo, il robot può navigare mantenendo una certa distanza dagli ostacoli, e ci si avvicinerà solo se necessario.
 - il *voxel layer*: popolato dallo Spatio-Temporal Voxel Layer (STVL), fornisce informazioni aggiuntive sulla percezione dell'ambiente tramite una griglia volumetrica tridimensionale.
- *Timed Elastic Band* (TEB) *controller* per l'esecuzione ottimizzata del piano generato.
- *Robot Localization* per sensor fusion e state estimation, tecnologia già trattata nella Sezione 3.1. Per la localizzazione in real-time viene utilizzato l'algoritmo *Adaptive Monte Carlo Localization* (AMCL).

Il risultato di questo progetto è uno stack di navigazione *robusto, affidabile e modulare*. Si tratta di una tecnologia affidabile perchè risolve i problemi di sicurezza relativi a robot mobili che superano i 2m/s attorno ad esseri umani grazie allo standard di comunicazione DDS di ROS 2; fornendo garanzie di recapito dei messaggi tra i nodi del robot allo stesso livello di quelle utilizzate per sistemi aerei e missilistici. É anche una tecnologia modulare, perché pensata per essere fortemente adattabile a robot diversi ed applicazioni diverse, con un behavior tree adattabile e server asincroni che possono rimanere attivi su altri core del processore. Anche gli algoritmi stessi applicati dai server possono essere modificati grazie al sistema di plugin.

4.2.2 QuadStack

Un'altra tecnologia per la navigazione studiata durante questo lavoro di tesi è QuadStack, uno stack di tecnologie per mapping, localizzazione e navigazione robusta ed efficiente per robot quadrupedi equipaggiati di sensori a basso costo [16]. Più nello specifico, mira a fornire queste funzionalità a robot quadrupedi equipaggiati di una videocamera RGBD ed un IMU, ed è stato implementato per l'adozione con robot quadrupedi di Unitree e MAB Robotics.

Il riferimento ai sensori a basso costo è importante per distinguere QuadStack da altre soluzioni, perché è in grado di ottenere risultati robusti con robot adottati principalmente su terreni accidentati ed irregolari, difficili da navigare, il che peggiora ulteriormente la qualità dei dati sensoriali ottenuti dai sensori di qualità minore. Per bilanciare la perdita di precisione, vengono adottate alcune tecniche, tra cui visual-inertial odometry, kinematic odometry migliorata tramite stime di contatto con il terreno e scansioni LiDAR stabilizzate tramite un IMU.

Il package risultante da questo lavoro è disponibile sul repository ufficiale² di GitHub.

4.2.3 Applicazione con Go2

Come per le operazioni di SLAM, la navigazione autonoma è una funzionalità molto comune; dunque, è stata utilizzata in simulazione con diversi modelli di robot durante l'intero periodo di studio. Nonostante ciò, è stato valutato rilevante riportare di seguito l'utilizzo del package QuadStack, illustrando alcuni dei file di launch disponibili.

1. Spawn del robot in simulazione. Ogni file di launch richiede come argomento il modello del robot quadrupede tra quelli supportati: `a1`, `go1`, `go2`, `silver_badger` e `honey_badger`. L'argomento `world`, invece, richiede il nome di uno dei mondi disponibili nella directory `/quadstack_gazebo/worlds`; è possibile inserirne altri in base alle necessità di simulazione, ricordando di inserire il path dei modelli nella variabile d'ambiente di Gazebo. Infine, come ogni spawn, è possibile specificare la posizione.

```
ros2 launch quadstack_bringup spawn_robot.launch.py \
  robot:<robot_name> \
  world:<world_name> \
  x_pose:<x> y_pose:<y> z_pose:<z>
```

2. Una volta spawnato il robot nel mondo, è possibile utilizzare il file di launch relativo alla funzionalità che si vuole simulare: semplice teleoperazione con `teleop.launch.py`, SLAM con `slam.launch.py`, o navigazione autonoma con `navigation.launch.py`. Con quest'ultimo, è possibile effettuare SLAM e navigation insieme, oppure navigation su una mappa già generata specificando l'argomento `map`.

```
ros2 launch quadstack_bringup odometry.launch.py
ros2 launch quadstack_bringup navigation.launch.py
↪ map:=absolute/path/to/map.yaml
```

²<https://github.com/dyumanaditya/quad-stack>

Nota: alcune di queste operazioni in pura simulazione possono essere molto costose dal punto di vista computazionale, quindi sono state eseguite tramite rosbags, in modo tale da avere solamente i messaggi da pubblicare, senza la necessità di calcolarne il contenuto in tempo reale.

4.3 Motion planning

All'interno dell'ambito di ricerca della robotica, il problema di spostare un oggetto da una posizione ad un'altra è un problema ricorrente: la navigazione, appena trattata nella sezione precedente, ne è un esempio. La risoluzione di questo tipo di problemi ricade nel contesto dell'*automazione*, occupandosi di trovare algoritmi in grado di calcolare i comandi da inviare ai giunti del robot per eseguire mansioni in cui deve muoversi totalmente o parzialmente, evitando ostacoli.

In questi algoritmi, la formalizzazione del problema è composta dalla rappresentazione del robot e del mondo in strutture bidimensionali o tridimensionali; e dalla soluzione, che è un percorso all'interno dello *spazio delle configurazioni*. Quest'ultimo può avere forme diverse in base al tipo di problema: in un semplice problema di navigazione autonoma, le configurazioni possibili sono tutte le coordinate (x, y) , mentre nel movimento di un braccio sono la rotazione (o traslazione per giunti prismatici) degli n giunti. Questo spazio può essere ridotto prima della pianificazione trovando lo spazio libero (*free space*), sottraendo le configurazioni non ammissibili a causa di collisioni con gli ostacoli. Nella realtà, questa operazione occuperebbe troppe risorse computazionali; la soluzione efficiente calcola la configurazione ottenuta tramite cinematica diretta (*forward kinematics*), ed un algoritmo ausiliario di *collision checking* determina se è valida.

In ROS 2, la libreria più utilizzata è MoveIt 2, piattaforma che implementa le soluzioni più avanzate per la risoluzione di motion planning, cinematica, manipolazione e percezione tridimensionale. Come tale, è stata utilizzata durante il lavoro di tesi sia per applicazioni pratiche di manipolazione con bracci, sia per lo sviluppo del progetto di integrazione finale.

4.3.1 MoveIt 2

La piattaforma di MoveIt 2 fornisce un intero sistema di calcolo dei movimenti con collision checking ed obstacle avoidance, formato da vari plugin che lo rendono flessibile e modulare. La sua architettura ROS, visibile in Figura 16, è costituita da un nodo principale, il nodo *move_group*, che si occupa di pianificazione tramite un sistema a plugin di pianificatori. La pianificazione è basata sulla prevenzione delle collisioni tramite la *planning scene* ed algoritmi di cinematica diretta ed inversa.

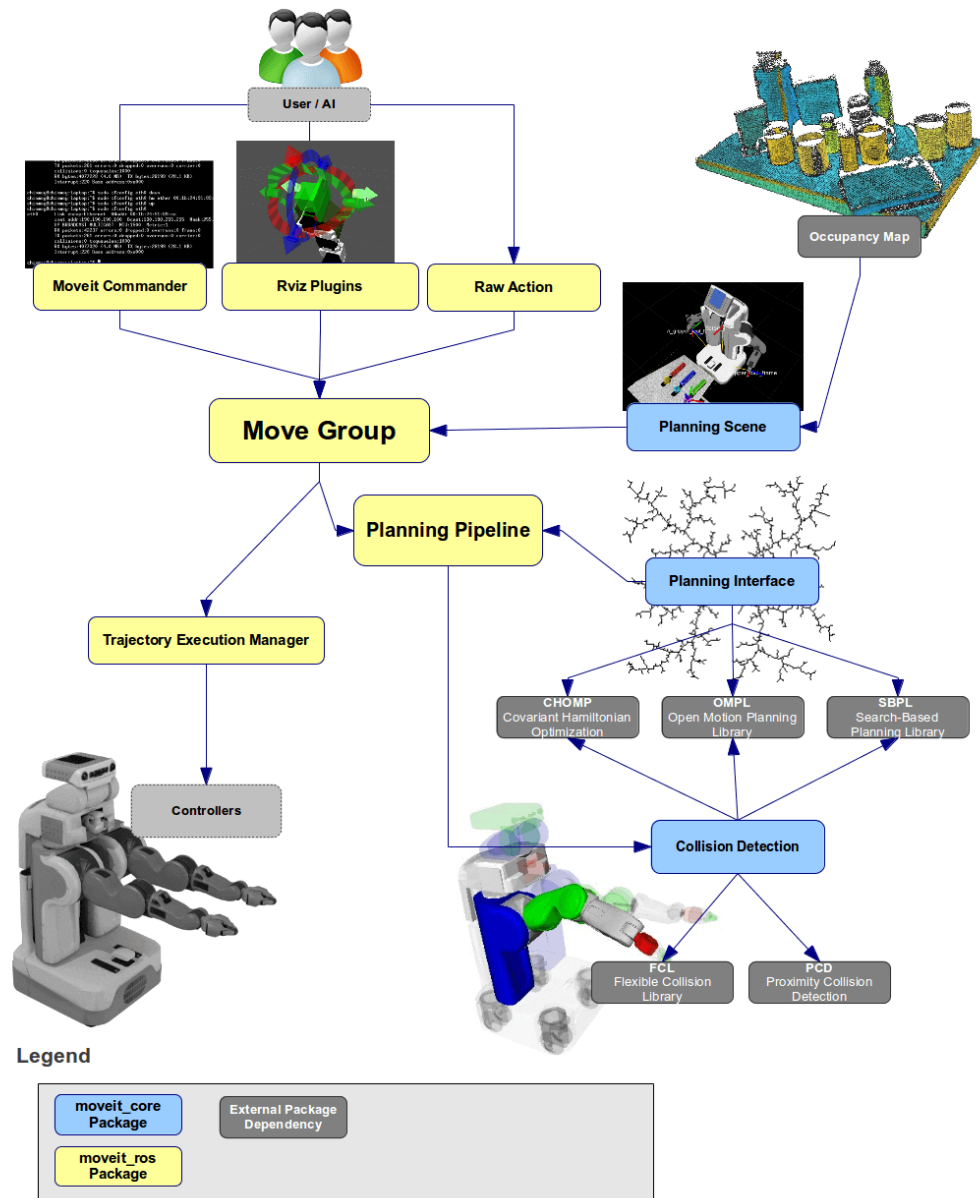


Figura 16: Diagramma della pipeline base di MoveIt 2.

Il nodo `move_group` è la parte centrale del sistema: raccoglie le percezioni tridimensionali del robot (generalmente una nuvola di punti `PointCloud`), le descrizioni del robot URDF e SRDF, ed il controller (generalmente un `JointTrajectoryController`), e mette a disposizione degli utenti diverse interfacce per eseguire operazioni di motion planning: la libreria `move_group_interface` per C++, la libreria `moveit_commander` per Python, oppure un'interfaccia grafica disponibile tramite il plugin di RViz. Il file in formato SRDF non è altro che la descrizione semantica del robot, generata solitamente tramite il MoveIt Setup Assistant, e contiene i raggruppamenti dei giunti, una lista di pose predefinite per ciascun gruppo ed altre configurazioni utili per MoveIt. La comunicazione del nodo con il robot avviene tramite topic e service ROS, tra cui:

- il topic `/joint_states` per determinare lo stato corrente del robot;
- i topic di tf2 per determinare le configurazioni globali del robot;
- il topic `/planning_scene` per leggere la geometria dell'ambiente circostante;
- l'action `FollowJointTrajectoryAction` per inviare comandi al controller del robot;

Nota: il nodo `move_group` non istanzia publisher per i topic né server per le action, quel compito è delegato al progettista del robot.

La *Planning Scene* è la rappresentazione del mondo circostante e del robot e viene gestita dal Planning Scene Monitor tramite un'interfaccia, resa anch'essa disponibile tramite le librerie C++ e Python di MoveIt 2 ed il plugin di RViz. La rappresentazione degli ostacoli può essere fornita a scelta tra mesh, forme tridimensionali primitive o octomap generate tramite la percezione del robot con i plugin *PointCloudOccupancyMapUpdater* e *DepthImageOccupancyMapUpdater*.

La pianificazione avviene tramite pianificatori differenti: MoveIt 2 usa plugin di motion planner, permettendo all'utente finale di scegliere l'algoritmo più appropriato; vengono messi a disposizione alcuni plugin predefiniti come i pianificatori di *Open Motion Planning Library* (OMPL), *Covariant Hamiltonian Optimization for Motion Planning* (CHOMP) ed il Plitz industrial motion planner. La richiesta di motion planning viene formata specificando la posizione che si vuole raggiungere ed alcuni limiti: di posizione, di orientamento, di velocità, anche limiti personalizzati. Infine, viene eseguita tramite un trajectory controller che genera una traiettoria correttamente limitata.

MoveIt Setup Assistant

Dal momento che il contesto trattato durante questo testo è caratterizzato dalla sua natura open source, è possibile trovare molti robot che dispongono già delle

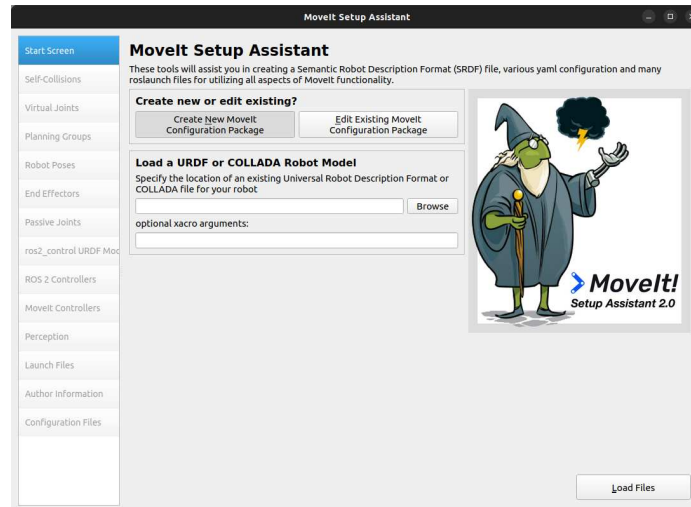


Figura 17: Schermata iniziale del MoveIt Setup Assistant.

configurazioni necessarie per operare con MoveIt 2; però, è possibile configurare un robot personale tramite un'interfaccia grafica intuitiva visibile in Figura 17, il *MoveIt Setup Assistant*. Tramite questo strumento avviabile tramite file di launch *setup_assistant.launch.py* del package *moveit_setup_assistant*, è possibile caricare la descrizione URDF del proprio robot e seguire la procedura di configurazione guidata e documentata.

La configurazione guidata permette di definire facilmente:

- la *matrice di auto-collisioni* per evitare di fare collision checking tra le parti del robot che è normale siano a contatto tra loro (es. due bracci collegati dallo stesso giunto).
- i *giunti virtuali* per ancorare la base del robot ad un frame specifico: generalmente un giunto fisso con world per robot statici ed un giunto planare con odom per robot mobili.
- i *planning group*: ciascun gruppo è composto da una serie di giunti e componenti, e rappresenta una sezione del robot in grado di svolgere task, come il braccio (costituito da molti giunti) o l'end effector (costituito da 2 o più dita).
- le pose predefinite dei gruppi, in modo tale da poter richiedere la pianificazione per la posa tramite il nome della posa e non le posizioni specifiche.
- la generazione del tag per i controller di *ros2_control* che verranno utilizzati da MoveIt per inviare comandi all'hardware.

- il setup per integrare la percezione tridimensionale del robot.

Una volta terminato, è convenzione salvare il package che verrà generato in una cartella nel src del proprio workspace denominata `<nome_robot>__moveit_config`.

4.3.2 MTC: MoveIt Task Constructor

Nel corso del tempo, MoveIt 2 è diventata la tecnologia principale per l'esecuzione di motion planning in ambienti open source, ed è stata estesa con il MoveIt Task Constructor (MTC): un framework che permette di strutturare operazioni complesse in maniera modulare. In questo contesto, ogni task è divisa in stages, che possono essere di diverso tipo e riutilizzate.

4.3.3 Applicazione con UR5 o Panda

L'applicazione pratica per lo studio e l'utilizzo di MoveIt 2 è stato un processo particolarmente semplice: è stato sufficiente creare l'ambiente di lavoro apposito come per ogni package ROS, clonando il repository ufficiale di MoveIt 2 come illustrato nella documentazione ufficiale. Una volta fatto, saranno disponibili una serie di file di launch appartenenti ai vari tutorial ufficiali, tra cui la coppia *demo.launch.py* e *run.launch.py* del package *moveit_task_constructor_demo*, che mostrano una semplice operazione di pick and place tramite un braccio robotico Panda come in Figura 18. L'utilità maggiore estratta da questo package, però, è stata la possibilità di esaminarne il codice, per studiare come scrivere un'operazione complessa di motion planning con le librerie MoveIt.

4.4 Object detection e pose estimation

Quando un robot ha la possibilità di avere una percezione visiva dell'ambiente esterno tramite sensori come una videocamera e di interagire con l'ambiente esterno tramite bracci controllabili con algoritmi di motion planning, l'ultima operazione rilevante è il riconoscimento di oggetti di interesse e la stima della loro posizione per poter assumere comportamenti dinamici. Questo avviene con la nota libreria di computer vision *OpenCV*, che mette a disposizione tutti gli strumenti necessari per lavorare con fonti video. In particolare, durante lo svolgimento del lavoro di tesi, è stata studiata una tecnica di object detection tramite ArUco Marker di OpenCV.³

Gli *ArUco markers* sono marcatori quadrati con codifica a matrice binaria rappresentata come dei quadrati bianchi su sfondo nero, come visibile in Figura 19. Un dizionario di marcatori viene definito come la lista di marcatori univoci disponibili;

³https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

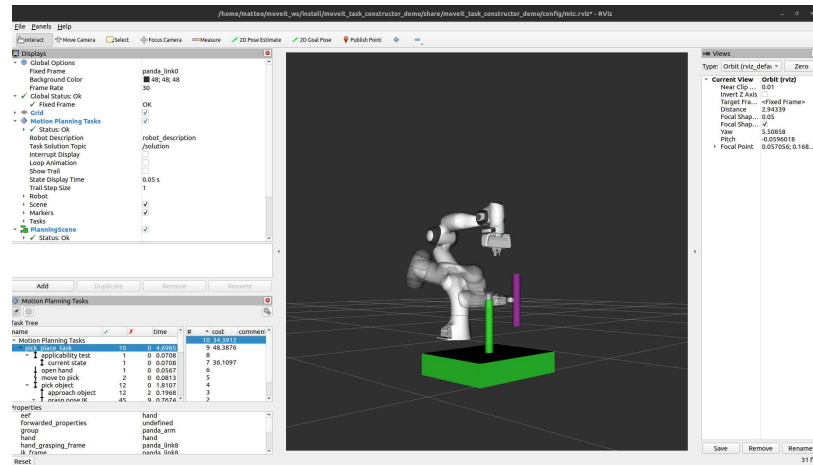


Figura 18: Visualizzazione dell'operazione di pick and place in RViz con un braccio Panda.

ciascun marcatore del dizionario possiede un ID univoco corrispondente al suo indice all'interno del dizionario (ossia, un dizionario da 250 marcatori contiene marcatori con ID da 0 a 249). Possono essere generati tramite OpenCV in dimensioni differenti, che determinano la composizione della matrice interna.

In ROS 2 Humble è possibile utilizzare questa tecnica di object detection e pose estimation tramite vari package disponibili online come *ros2_aruco* oppure scrivendo il proprio nodo in grado di iscriversi al topic su cui vengono pubblicate le immagini della videocamera, elaborarle con OpenCV, e pubblicarne il risultato su un altro topic.

4.4.1 Applicazione con TurtleBot Home Service Challenge

L'applicazione pratica studiata per le operazioni di object detection è quella delle *Home Service Challenge* di Turtlebot. In questi scenari proposti da Robotis, viene presentato un package di simulazione con un TurtleBot 3 Waffle Pi integrato ad un braccio robotico OpenMANIPULATOR, con varie task da eseguire all'interno di una mappa simile ad un contesto casalingo (Figura 20). All'interno del mondo sono disposti alcuni marcatori ArUco la cui posizione globale è nota (riportata nel file scenario.yaml), e viene usata la funzionalità di riconoscimento visivo di questi marcatori per migliorare la precisione dei movimenti del robot quando si trova nei loro pressi. L'obiettivo in ciascuno di questi scenari è spostare alcuni oggetti e riporli in parti differenti della mappa, sfruttando la navigazione autonoma di Navigation 2, il movimento del braccio robotico tramite motion planning di MoveIt 2 e riconoscimento degli oggetti tramite marcatori ArUco.

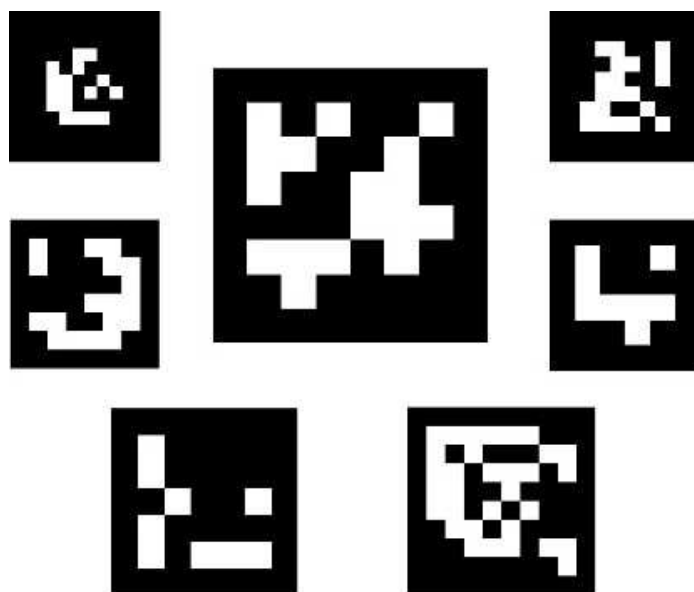


Figura 19: Esempi di marcatori ArUco.

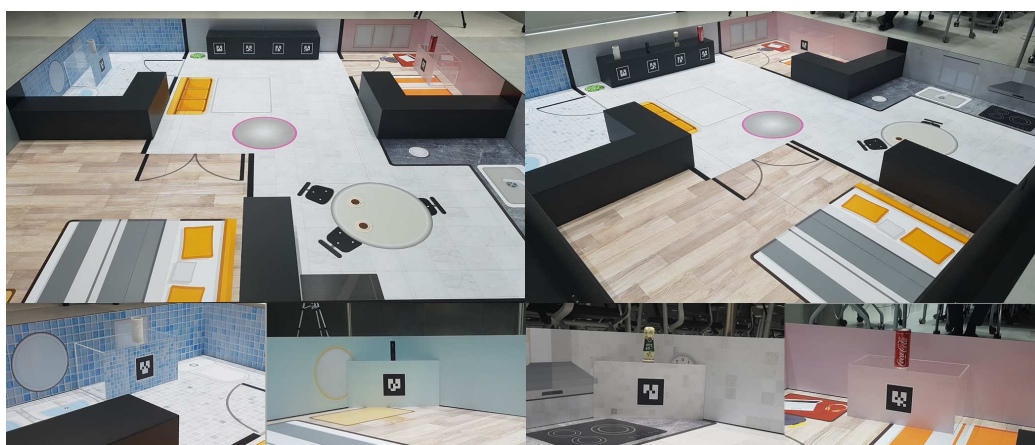


Figura 20: Rappresentazione dettagliata della mappa casalinga della Home Scervice Challenge, con i marcatori ArUco posizionati sul ripiano degli oggetti da spostare.

Capitolo 5

Integrazione delle tecnologie

Per terminare il lavoro di tesi è stato valutato pertinente aggiungere un'implementazione delle tecnologie ed operazioni trattate nei capitoli precedenti. In questo capitolo viene illustrato l'approccio al lavoro per questo progetto ed i suoi risultati, partendo dalla progettazione allo sviluppo, con alcune considerazioni finali e illustrazioni.

5.1 Progettazione dello scenario completo

L'obiettivo del progetto è quello di sviluppare un package ROS 2 che dimostri l'acquisizione delle competenze di sviluppo di applicazioni per agenti robotici tramite le tecnologie studiate nel corso di questo lavoro di tesi. A seguito di un'attenta valutazione, è stato scelto di implementare uno scenario di automazione tramite un TurtleBot 3 con OpenMANIPULATOR per un'operazione di pick and place con navigazione autonoma. L'OpenMANIPULATOR è un braccio robotico di natura open source sia nelle sue componenti software, sia nelle sue componenti hardware¹. L'operazione di pick and place, invece, è un'operazione di base per robot mobili dotati di braccia e funzionalità di motion planning con collision avoidance.

Lo scenario è stato ideato come segue: il robot deve navigare nella direzione di un oggetto, raccoglierlo (*fase di pick*), spostarsi in un'altra zona, e riporlo (*fase di place*); il tutto dovrebbe essere fatto *in autonomia*, minimizzando la necessità di input da parte di operatori esterni. Più in dettaglio:

- Il robot scelto è un TurtleBot 3 con OpenMANIPULATOR, grazie alla presenza di un modello per simulazione già testato in precedenza ed una configurazione con MoveIt 2 pre-esistente per il controllo del braccio.

¹Lista dei componenti in formato STL: <https://cad.onshape.com/documents/9442f03bd8ccac084fda9dd3/w/039e8dbd53e0782540ea5b0d/e/6f08aa8ac3d3e5b3054f7782>

- Il mondo e l'oggetto: per l'ambiente è stato costruito un mondo di Gazebo Classic ad hoc, sfruttando il modello di TurtleBot World modificato per fare spazio ad oggetti di scena quali l'oggetto da raccogliere (modellato come un semplice paletto di 30cm di altezza) e le due zone di raccolta e deposito, rispettivamente raffigurate come due tappeti circolari di colore verde e rosso.
- La navigazione viene eseguita tramite Navigation 2, già presente nei package del robot di base (un TurtleBot 3 Waffle), ma con una configurazione differente per aumentare la precisione, riducendo la tolleranza tra la posizione raggiunta e quella fornita come obiettivo (goal). Nonostante la configurazione più precisa, è stato valutato necessario introdurre un nodo aggiuntivo in grado di parcheggiare il robot nei pressi dell'oggetto da raccogliere in maniera più precisa: in questo modo si ha maggiore controllo sulla distanza da raggiungere dall'oggetto affinché si trovi all'interno del raggio d'azione del braccio.
- Il riconoscimento dell'oggetto e la stima della sua posizione sono fatte tramite marcatori ArUco e OpenCV, con il package `ros2_aruco` ed un marcatore inserito manualmente nel modello SDF dell'oggetto. Oltre al package, si è palesata la necessità di sviluppare un nodo per gestire la posizione del marcatore in maniera più adeguata rispetto a quanto fornito dal package, in particolare rispetto alla rotazione ed al frame di riferimento dei messaggi.
- Le operazioni di pick e di place vengono effettuate tramite MoveIt 2 e la configurazione pre-esistente del robot, in un nodo sviluppato personalmente. L'operazione di pick è stata implementata in maniera dinamica: la posa del gripper per afferrare l'oggetto viene calcolata in base alla stima della posizione ottenuta con la funzionalità descritta nel punto precedente.

5.2 Sviluppo

Lo sviluppo del progetto ha seguito grosso modo i passi elencati nella sezione precedente. La prima task compiuta è stata la **creazione del mondo** visibile in Figura 21, composto da un modello di TurtleBot 3 world il cui file SDF è stato modificato rimuovendo alcune colonne per consentire l'aggiunta degli oggetti di scena. Tra questi, due aree circolari create tramite il model editor di Gazebo Classic ed un oggetto generico la cui descrizione SDF è stata scritta manualmente; al suo interno, si trova un marcatore ArUco modellato tramite una mesh creata in Blender e texturizzata con un marcatore generato tramite il package `ros2_aruco`, per poi essere esportata in formato STL (*STereoLithography*) con la versione 4.0 del software, visibile in Figura 22.

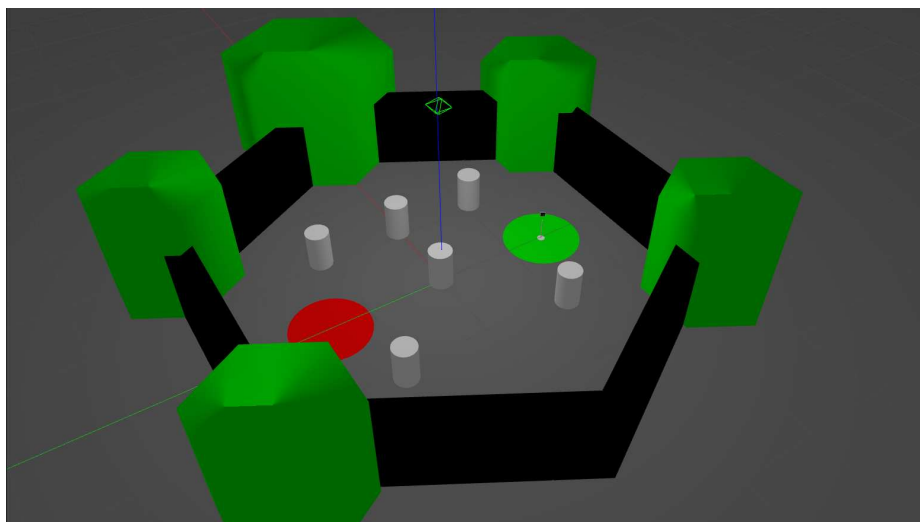


Figura 21: Mondo creato per il progetto.

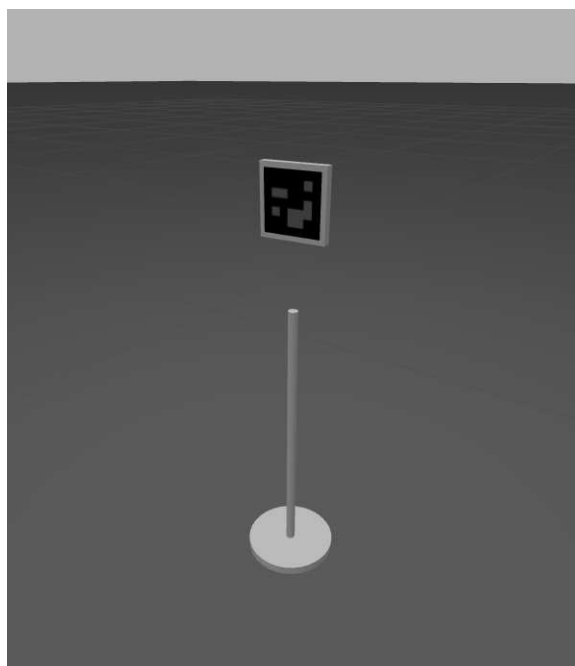


Figura 22: Oggetto da afferrare con il relativo marcatore ArUco.

Successivamente, è stato necessario **adeguare il modello** del TurtleBot 3 Waffle ad un'operazione di pick and place con object detection tramite videocamera: in questi contesti, infatti, è solito seguire un'impostazione *eye-to-hand* oppure un'impostazione *eye-in-hand*. Nell'impostazione *eye-to-hand* la videocamera è fissata in una posizione statica ed esterna al robot, fornendo una visuale globale e molto ampia dell'ambiente; è meno precisa ma consente operazioni più rapide perchè può calcolare immediatamente le posizioni target. Nell'impostazione *eye-in-hand*, la videocamera è montata in prossimità dell'end effector, consentendo una visuale ravvicinata e più precisa adatta per manipolazioni complesse, ma più lenta a causa della necessità di spostare il braccio per ottenere una prima visione del target. Per questo lavoro è stata scelta un'impostazione *eye-to-hand* spostando la videocamera del robot in alto ed indietro di 40cm, per consentire una visuale più ampia e sfruttare le trasformazioni di TF per la stima della posizione del marcatore. A causa della descrizione modificata del robot, è stato necessario adattare i file di launch originali per il setup del robot completo, tra cui i file `gazebo.launch.py` e `base.launch.py` del package `turtlebot3_manipulation_gazebo`.

La **stima della posizione** viene eseguita direttamente dal package `ros2_aruco` a seguito di una semplice configurazione del file `config/aruco_parameters.yaml` per specificare dimensione del marcatore (necessaria per la stima della posizione), tipo di dizionario ArUco utilizzato, ed i topic con i dati della videocamera e dove vengono pubblicate le immagini. La posa stimata dal package utilizza come frame di riferimento quello della videocamera ed un sistema di coordinate ottico, dunque è stato sviluppato un nodo intermedio `read_marker_position` che rileva la posizione del marcatore con ID corretto, lo ruota per adeguarlo al sistema di coordinate standard di ROS 2 e ne pubblica la trasformazione verso il frame `base_footprint`, che è anche il frame di riferimento utilizzato da MoveIt 2 per muovere l'end effector.

La **navigazione** iniziale per raggiungere una posizione in cui il marcatore sia visibile, come anche la navigazione con l'oggetto in mano, viene eseguita tramite Navigation 2 mediante il package di navigazione originale `turtlebot3_manipulation_navigation2` con i parametri ottenuti dalla repository della Home Service Challenge per migliorare la precisione della navigazione al costo di qualche comportamento di recovery aggiuntivo in alcuni casi. La mappa del mondo è stata generata con il package originale di mapping `turtlebot3_manipulation_cartographer`.

Durante la fase di prototipizzazione e testing dei package pre-esistenti si è notata una certa imprecisione del package Navigation 2 per parcheggiare il robot in prossimità dell'oggetto da afferrare: trattandosi di un robot di piccole dimensioni (circa 30cm×30cm) ed un braccio robotico con un raggio d'azione limitato, un errore generalmente irrilevante di qualche centimetro è in grado di compromettere la

robustezza dell'operazione. Per risolvere questo problema è stato sviluppato un nodo `follow_marker_pose` che legge la posizione del marcatore pubblicata dal nodo `read_marker_position` e pubblica comandi di velocità sul topic `/cmd_vel` per parcheggiare il robot al massimo a 30cm di distanza e con una tolleranza di circa 2 gradi di rotazione. Dal momento che il package per il rilevamento del marcatore ArUco pubblica solamente quando vi è un marcatore visibile, si utilizzano i timestamp degli header dei messaggi contenenti la posizione per assicurarsi di non stare seguendo un marcatore non più visibile.

Le operazioni di **pick** e di **place** vengono eseguite rispettivamente dai nodi `execute_pick.cpp` e `execute_place.cpp`. Eseguono le operazioni direttamente all'interno di una funzione `main()` all'opposto del solito approccio object-oriented, in modo tale da esprimere più facilmente la natura sequenziale e sincrona delle operazioni da svolgere. Innanzitutto, dal momento che l'operazione di pick and place viene divisa in due intervalli di tempo separati, è stato scelto di non utilizzare il MoveIt Task Constructor, preferendo un'implementazione diretta tramite l'API per C++ di MoveIt 2. L'operazione di pick viene sincronizzata con l'ambiente di simulazione pubblicando in tempo reale un collision object nella planning scene di MoveIt tramite la posizione del marcatore, operazione delegata al nodo `read_marker_position` che mantiene al suo interno un planning scene monitor per aggiungere l'oggetto man mano che la sua posizione cambia in relazione al robot che si sposta. Lo stesso nodo espone un ROS service per ottenere la posizione del marcatore in un determinato momento, che viene utilizzato dal nodo `execute_pick` per l'implementazione di un movimento di grasp dinamico rispetto alla posizione dell'oggetto da afferrare. Il nodo `execute_place`, invece, prende la posizione corrente e la usa per estendere leggermente il braccio ed aprire il gripper, rilasciando l'oggetto.

Dal momento che si tratta di uno scenario in simulazione, è stato ritenuto adeguato semplificare la parte di modellazione dei coefficienti d'attrito dell'oggetto per permettere al robot di afferrarlo, con un semplice plugin IFRA LinkAttacher [17] che permette di attaccare il link di un oggetto ad un altro, come se ne fosse il figlio nell'albero di TF; in questo modo, una volta eseguita l'operazione di chiusura del gripper con MoveIt 2, si utilizza il ROS service esposto dal plugin per attaccare l'oggetto al gripper del robot, ottenendo essenzialmente lo stesso risultato di una dispendiosa operazione di fine tuning dei valori di attrito e forma dell'oggetto.

Infine, i **file di launch** separano l'esecuzione in due fasi:

- Una fase di **setup** con `complete_scenario_with_object_detection.launch.py`: in questo file vengono eseguiti i file di launch per l'avvio del nodo `move_group` di MoveIt 2, per lo spawn del robot in Gazebo, per l'inizializzazione dei nodi di Navigation 2 e per l'avvio dell'object detection. In questo modo, viene preparato l'ambiente per l'esecuzione delle operazioni, che potrebbero necessitare di alcuni requisiti come la presenza di tutte le trasformazioni da map ai link del robot.

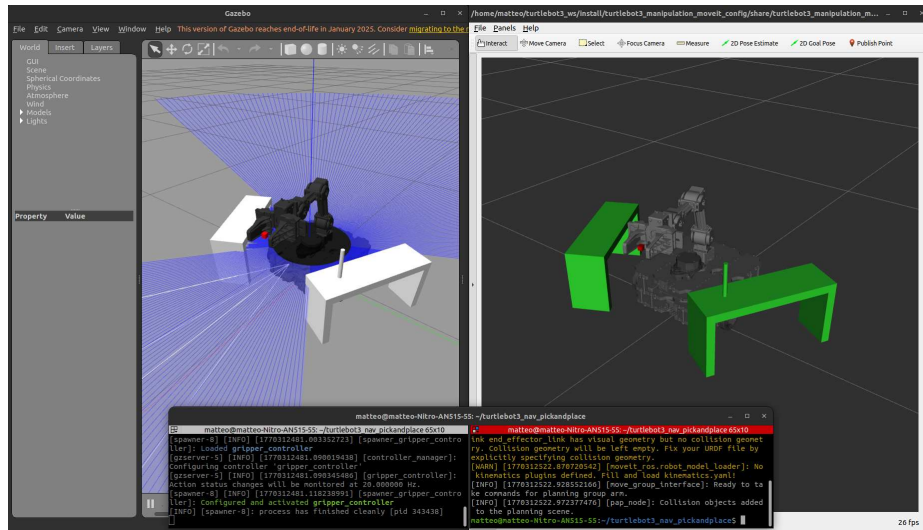


Figura 23: Operazione di pick and place statica facente parte di un prototipo prodotto durante le fasi di test.

- Una fase di **esecuzione** con `execute_complete_scenario_with_object_detection.launch.py`: questo file utilizza una lista di `EventHandler` e `TimerAction` per l'esecuzione sequenziale dei nodi adibiti alle subtask dell'operazione. Gli `EventHandler` più utilizzati sono `OnProcessExit`, il ch  ha portato la necessit  di gestire il termine di esecuzione di ciascuno dei nodi facenti parte delle sequenza di subtask.

Prototipi intermedi

Il codice e le risorse prodotti nel corso di questo lavoro di tesi non sono limitati a quelli appartenenti a questo progetto: sono infatti presenti nodi per l'esecuzione di pick and place statici con `pick_and_place.launch.py` come visibile in Figura 23 ed un'operazione di pick and place pi  dinamica con `complete_scenario.launch.py`, che presenta una fase di navigazione ma senza implementare l'object detection: l'imprecisione di questo risultato ha portato allo sviluppo del codice discusso fin'ora.

5.3 Applicazione con `turtlebot3_nav_pickandplace`

Il codice   stato inserito in un package reso disponibile in un repository GitHub², e pu  essere facilmente testato seguendo le indicazioni riportate nel file `README.md`;

²https://github.com/MatteoVignaga/Turtlebot3_Nav_PickAndPlace/tree/main

si riporta di seguito una rapida guida all'utilizzo per completezza:

1. Creazione del workspace e clonare il repository ed i package accessori: IFRA_Linkattacher, ros2_aruco, turtlebot3_simulations, turtlebot3_manipulation.

```
mkdir -p workspace_name/src
cd workspace_name/src
git clone https://github.com/MatteoVignaga/Turtlebot3_Nav_PickAndPlace.git
git clone -b humble
↪ https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
git clone -b humble-devel
↪ https://github.com/ROBOTIS-GIT/turtlebot3_manipulation.git
git clone https://github.com/IFRA-Cranfield/IFRA_LinkAttacher.git
git clone -b opencv_4.7 https://github.com/JMU-ROBOTICS-VIVA/ros2_aruco.git
```

2. Installare le dipendenze con rosdep.

```
rosdep init
rosdep install --from-paths src -y --ignore-src
```

3. Compilare con Colcon.

```
cd ~/nome_workspace
colcon build --symlink-install
```

4. Source dell'overlay.

```
source install/setup.bash
```

5. Esecuzione del setup.

```
ros2 launch tb3_nav_pick_and_place
↪ complete_scenario_with_object_detection.launch.py
```

6. Esecuzione dello scenario di automazione.

```
ros2 launch tb3_nav_pick_and_place
↪ execute_complete_scenario_with_object_detection.launch.py
```

Nota: è possibile eseguire le subtask individualmente tramite i relativi nodi, la struttura del file di launch dell'esecuzione ne riporta accuratamente l'utilizzo.

Il risultato del progetto, visibile in Figura 24 e tramite una video-dimostrazione³, è un package che dimostra le potenzialità delle tecnologie di robotica open source per l'automazione di operazioni ripetitive, anche complesse. L'esecuzione completa risulta robusta ed affidabile, ma è importante tenere presente che a causa della natura di simulazione si tratta di caratteristiche legate alle prestazioni del dispositivo su cui

³<https://youtu.be/NwpsNIYv7PI>

viene eseguito: una capacità computazionale ridotta, infatti, porta alla perdita di molti messaggi dei sensori, risultando in una risoluzione della percezione ridotta e dunque maggiori possibilità di fallimento. Ad ogni modo, in tal caso è sufficiente terminare l'esecuzione dei due file di launch e ri-eseguire i due comandi per riprovare.

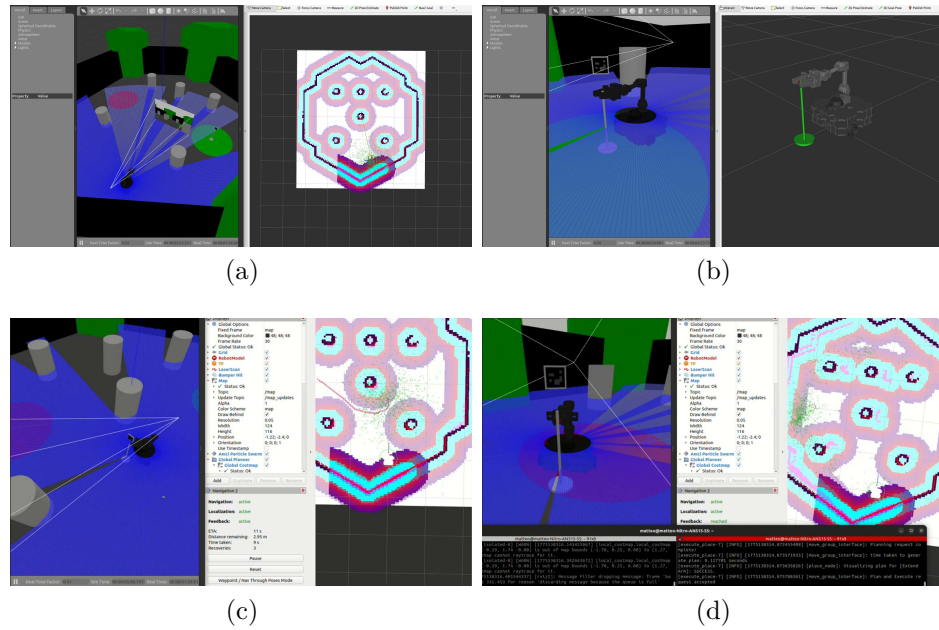


Figura 24: Dimostrazione grafica dell'esecuzione: spawn iniziale (a), parcheggio in prossimità dell'oggetto (b), navigazione autonoma con l'oggetto (c), riposizione dell'oggetto nell'area adibita (d).

Capitolo 6

Conclusioni

Questo lavoro di tesi ha coinvolto un'estesa gamma di concetti e tecnologie utilizzate nella robotica, permettendo non solo l'acquisizione delle competenze di base per operare con robot mobili, ma anche lo sviluppo di un sistema di automazione per una generica mansione di movimento di oggetti, comune in ambiti industriali e casalinghi. Tutte le tecnologie trattate sono open source e permettono di migliorare l'accessibilità a questo tipo di applicazioni sempre più rilevanti nei contesti industriali, agricoli, urbani e persino casalinghi.

Possibili sviluppi

Mantenendo la natura open source comune a tutte le tecnologie utilizzate, si potrebbe considerare di estendere questo lavoro di studio ed analisi dell'ambiente di robotica open source esplorando ulteriori soluzioni aperte al pubblico, tra cui quelle elencate di seguito.

- RoboWare Studio¹, un ambiente di sviluppo integrato (IDE) sviluppato specialmente allo scopo di gestione di workspace ROS. Al suo interno vi sono molti strumenti e funzionalità che migliorano la cosiddetta "quality of life" (qualità della vita) dello sviluppo di pacchetti per ROS 2.
- Foxglove², uno strumento di visualizzazione consolidato nell'ambiente di robotica che fornisce una solida alternativa ad RViz, con una capacità di analisi dei dati più estesa e supporto per una maggiore varietà di tipi di dati.
- Isaac Sim³, simulatore sviluppato da NVIDIA per simulazioni di robot. È un simulatore pensato per sviluppo professionale, come si può dedurre dai requisiti

¹<https://github.com/TonyRobotics/RoboWare-Studio.git>

²<https://foxglove.dev/>

³<https://github.com/isaac-sim/IsaacSim.git>

hardware per il suo utilizzo (come minimo un processore i7, 32GB di RAM ed una scheda grafica NVIDIA GeForce RTX4080 con 16GB di VRAM).

Bibliografia

- [1] MetroRobots. A guide to understanding launch files in ros 1 and ros 2. https://github.com/MetroRobots/rosetta_launch.
- [2] ROS Developers. Index of ros enhancement proposals (reps). <https://ros.org/reps/rep-0000.html>, 2000.
- [3] Ken Conley. Rep purpose and guidelines. <https://ros.org/reps/rep-0001.html>, 2010.
- [4] ROS Developers. Rviz user guide. <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html>.
- [5] Frank C. Park Kevin M. Lynch. *Modern Robotics*. Cambridge University Press, 2017.
- [6] ROBOTIS. Turtlebot 3 features. <https://emanual.robotis.com/docs/en/platform/turtlebot3/features/#specifications>.
- [7] Motilal Agrawal and Kurt Konolige. Real-time localization in outdoor environments using stereo vision and inexpensive gps. volume 3, pages 1063–1068, 01 2006.
- [8] T. Moore and D. Stouch. A generalized extended kalman filter implementation for the robot operating system. In *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, July 2014.
- [9] Mordechai Ben-Ari and Francesco Mondada. *Mapping*, pages 141–163. Springer International Publishing, Cham, 2018.
- [10] ROBOTIS. Turtlebot 3 slam. <https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/>.
- [11] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2d lidar slam. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, 2016.

- [12] Steve Macenski and Ivona Jambrecic. Slam toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021.
- [13] Danial Nakhaeinia, S Hong Tang, SB Mohd Noor, and O Motlagh. A review of control architectures for autonomous navigation of mobile robots. *International Journal of the Physical Sciences*, 6(2):169–174, 2011.
- [14] Steve Macenski, Francisco Martín, Ruffin White, and Jonatan Ginés Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2718–2725, 2020.
- [15] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and ai, July 2018.
- [16] Dyuman Aditya, Junning Huang, Nico Bohlinger, Piotr Kicki, Krzysztof Walas, Jan Peters, Matteo Luperto, and Davide Tateo. Robust localization, mapping, and navigation for quadruped robots, 2025.
- [17] IFRA-Cranfield. Gazebo-ros2 link attacher. https://github.com/IFRA-Cranfield/IFRA_LinkAttacher, 2023.



Progetto sviluppato presso il Laboratorio di Sistemi Intelligenti Applicati (AIS Lab)
<http://ais-lab.di.unimi.it>